

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

МАТЕМАТИЧНИЙ ФАКУЛЬТЕТ

Кафедра програмної інженерії

КУРСОВА РОБОТА

з Архітектура та проектування програмного забезпечення

(назва дисципліни)

на тему: «Проектування та реалізація інформаційної системи
«Читального залу»»

Студентки 3 курсу, групи 6.1213-2пi
спеціальності 121 інженерія програмного забезпечення
(шифр і назва спеціальності)

Р.Ю. Ямацїта

(ініціали та прїзвище)

Керівник доцент кафедри програмної
їнженерїї, к.ф.-м.н., Мильцев О.М.
(посада, вчене звання, науковий ступїнь, прїзвище та ініціали)

Національна шкала: _____

Кількість балів: _____

Оцїнка ECTS: _____

Члени комісії:

(підпис)

(ініціали та прїзвище)

(підпис)

(ініціали та прїзвище)

(підпис)

(ініціали та прїзвище)

Запорїжжя – 2026

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

Факультет математичний

Кафедра програмної інженерії

Дисципліна Архітектура та проектування програмного забезпечення

Спеціальність 121 інженерія програмного забезпечення
(шифр і назва)

Освітня програма Програмна інженерія

З А В Д А Н Н Я
НА КУРСОВУ РОБОТУ СТУДЕНТЦІ

Ямащіті Рікаенжелі Юкінобівні

(прізвище, ім'я та по-батькові)

1. Тема роботи «Проектування та реалізація інформаційної системи
«Читального залу»»

2. Строк здачі студентом закінченої роботи

3. Вихідні дані до роботи 1. Постановка задачі.
2. Перелік літератури.

4. Зміст роботи (перелік питань, які потрібно розробити)

1. Аналіз предметної області, аналіз та специфікація вимог до ПЗ.

2. Визначення архітектури ПЗ, проектування БД та вебзастосунку.

3. Реалізація та тестування ПЗ.

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

ERM, діаграми UML, скріншоти вебзастосунку, презентація за темою доповіді

6. Дата видачі завдання 06.03.2026

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів курсової роботи	Термін виконання етапів роботи	Примітка
1.	Розробка плану роботи		
2.	Збір вихідних даних		
3.	Обробка методичних джерел		
4.	Розробка першого розділу		
5.	Розробка другого розділу		
6.	Оформлення курсової роботи		
7.	Захист курсової роботи		

Студент

(підпис)

Р.Ю. Ямашіта

(ініціали та прізвище)

Керівник роботи

(підпис)

О.М. Мильцев

(ініціали та прізвище)

РЕФЕРАТ

Курсова робота «Проектування та реалізація інформаційної системи «Читального залу»»: 73с., 72 рис., 2 табл., 23 джерела.

АППЗ, АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ, СУБД, СИСТЕМА УПРАВЛІННЯ БАЗАМИ ДАНИХ, ДАНІ, БАЗИ ДАНИХ, ER-ДІАГРАМА, ER-МОДЕЛЬ, UML, POSTGRESQL, WEB, HTML, CSS, REACT, NODEJS, JAVASCRIPT, EXPRESS, MICROSOFT VISUAL STUDIO CODE

Об'єкт дослідження — процес проектування та створення вебзастосунку для читання азійських романів.

Предмет дослідження — методи проектування вебзастосунку та моделі структур, поведінки.

Мета роботи — аналіз предметної області, проектування та розробка вебзастосунку для читання азійських романів.

Методи дослідження — методи проектування, методи програмування вебзастосунків.

Перший розділ присвячений аналізу предметної області, демонстрації існуючих рішень з даної предметної області, аналізу вибору засобів розробки та опису функціональних вимог та вимог інтерфейсу. У другому розділі, що присвячений проектуванню, розглянуто ER-модель та UML-діаграми. У третьому розділі описано проектування та реалізацію клієнтської частини вебдодатка мовами HTML5 та CSS3 з використанням бібліотеки React. Також висвітлено розробку серверної частини на платформі Node.js із застосуванням фреймворку Express.js. В додатках наведено програмну реалізацію.

ЗМІСТ

Завдання на курсову роботу	2
КУРСОВА РОБОТА	7
РЕФЕРАТ	10
ВСТУП	6
1 АНАЛІЗ ТА СПЕЦИФІКАЦІЯ ВИМОГ	8
1.1 Аналіз предметної області	8
1.2 Вимоги до програмного забезпечення	12
1.3 Вибір засобів розробки	15
2 ПРОЄКТУВАННЯ СИСТЕМИ	6
2.1 ER-модель даних	6
2.2 Діаграма прецедентів	10
2.3 Аналіз прецедентів за допомогою діаграм послідовностей та кооперацій	34
2.4 Діаграма класів	36
2.5 Діаграма діяльності	39
2.6 Діаграма компонентів	41
2.7 Діаграма розгортання	42
3 РОЗРОБКА ТА ТЕСТУВАННЯ СИСТЕМИ	43
3.1 Загальний опис розробки інтерфейсу	43
3.2 Нетривіальні рішення	44
3.3 Демонстрація застосунку	46
3.4 Тестування застосунку	68
ВИСНОВКИ	70
ДОДАТОК А	73
Програмна реалізація	73

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАКИ

ПЗ Програмне забезпечення

БД Бази даних

ВСТУП

Сюжети китайської та японської культур дуже різноманітні. Цього було достатньо для неймовірного розвитку спільноти фанатів азійських романів, які є дуже популярними у поточні часи. Серед україномовної аудиторії чимало тих, хто хотів би мати можливість прочитати такі твори в перекладі онлайн, при цьому маючи можливість коментувати книгу, додавати її до себе в уподобання та знаходити потрібний контент за жанрами або назвою, шукати авторів та інші їх романи. Важливо також отримати можливість отримувати інформацію про обрану книгу та статус завершення її перекладу.

Отже, поточною задачею є проєктування та подальше створення окремого вебзастосунку для читання азійських романів.

Метою курсової роботи є проєктування та розробка вебзастосунку для зберігання інформації про книги, їх жанри та авторів з можливістю читати твори та ділитися своєю думкою в коментарях.

Задачі, які необхідно розв'язати для досягнення поставленої мети:

- проаналізувати предметну область «Азійські романи»;
- проаналізувати існуючі рішення з даної предметної області;
- проаналізувати обрані засоби розробки (HTML та CSS з використанням бібліотеки React, JavaScript для написання серверу);
- сформулювати вимоги до вебзастосунку;
- реалізувати базу даних засобами PostgreSQL;
- спроектувати вебзастосунок згідно вимог;
- розробити вебзастосунок за допомогою обраних засобів;
- протестувати вебзастосунок.

Об'єкт дослідження — процес проєктування та створення вебзастосунку для читання азійських романів.

Предмет дослідження — методи проєктування вебзастосунку та моделі структур, поведінки.

Методи дослідження — методи проєктування, методи програмування вебзастосунків.

Структурно курсова робота складається з таких компонентів: вступ, 3 розділи, висновки, 1 додаток, перелік посилань із 23 найменувань.

Перший розділ присвячений аналізу предметної області, демонстрації існуючих рішень з даної предметної області, виділенню переваг та недоліків, аналізу вибору засобів розробки та опису функціональних вимог та вимог інтерфейсу застосунку. У другому розділі, що присвячений проєктуванню, розглянуто ER-модель та UML-діаграми (діаграма прецедентів (для пояснення можливостей використання вебзастосунку), діаграма послідовностей (для ілюстрації алгоритмів взаємодії окремих елементів системи), діаграма класів (для пояснення внутрішньої архітектури програмної частини), діаграма діяльності (для візуалізації алгоритму обраного процесу), діаграма компонентів (для відображення фізичної структури коду), діаграма розгортання (для пояснення фізичного розміщення/розгортання системи)). У третьому розділі описано проєктування та реалізацію клієнтської частини вебдодатка мовами HTML5 та CSS3 з використанням бібліотеки React. Також висвітлено розробку серверної частини на платформі Node.js із застосуванням фреймворку Express.js. В додатках наведено програмну реалізацію.

1 АНАЛІЗ ТА СПЕЦИФІКАЦІЯ ВИМОГ

1.1 Аналіз предметної області

Предметною областю є азійські романи, які мають неповторний стиль, що приваблює аудиторію різноманітної статі та віку. Твори здатні змусити зацікавлених поринути у неймовірні витoki культури Китаю та Японії, а оскільки вони дуже відрізняються від європейської — це викликає сильний інтерес зокрема українського суспільства. На тлі загального піднесення популярності культури Далекого сходу така тенденція є цілком передбачуваною подією.

З приводу середовища для розгортання застосунку обрано саме вебплатформу через її зручність, швидкість доступу та зниження бажання користувачів завантажувати застосунки на пристрої. Для курсової обрано створення вебзастосунку також за потреби опанувати навички на практиці та отримати персональний досвід.

Перед початком проєктування важливо проаналізувати вже існуючі подібні рішення для розуміння приблизного вигляду майбутнього застосунку та прикладу можливих недоліків або переваг, що надають конкуренти.

Проаналізуємо вебсайт **baka.in.ua** [1] з обраною предметною областю. Головна сторінка містить загальний блок з коротким описом та можливістю перейти до цікавих розділів. Значною перевагою є наявність такої навігації. Є також можливість зареєструватися та надіслати повідомлення (див. рис. 1.1).

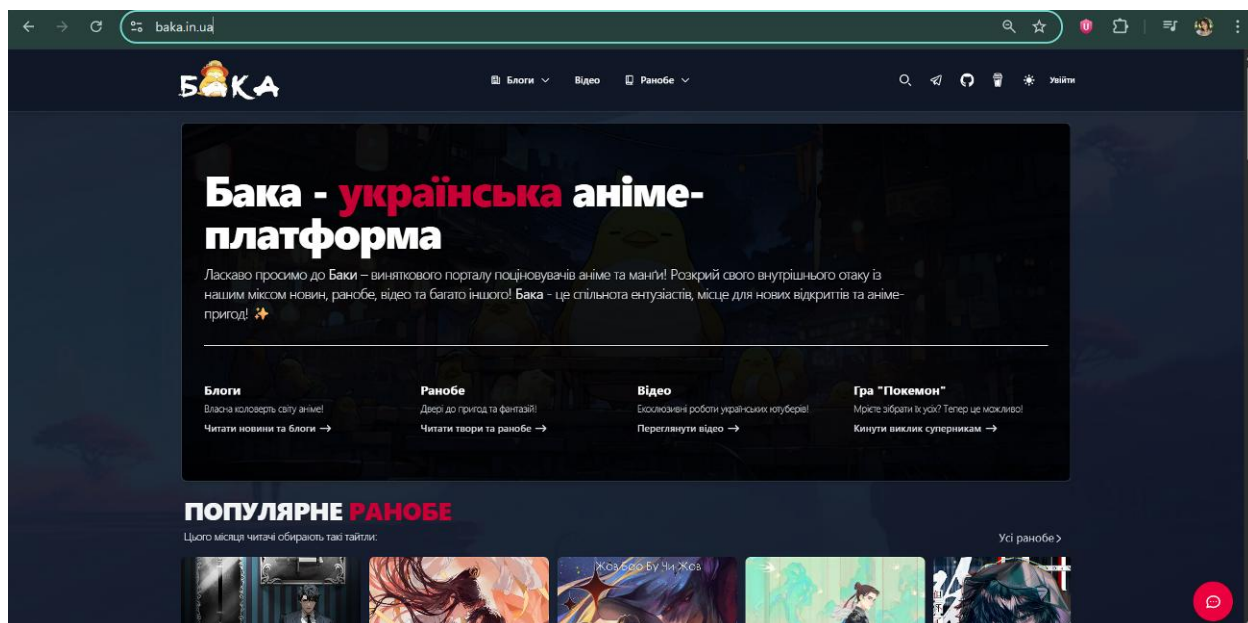


Рисунок 1.1 – Головна сторінка baka.in.ua

На головній сторінці також є блок новин про події, які відбулися у спільноті. Кожна стаття з голосним заголовком, що можна прочитати та прокоментувати (див. рис. 1.2).

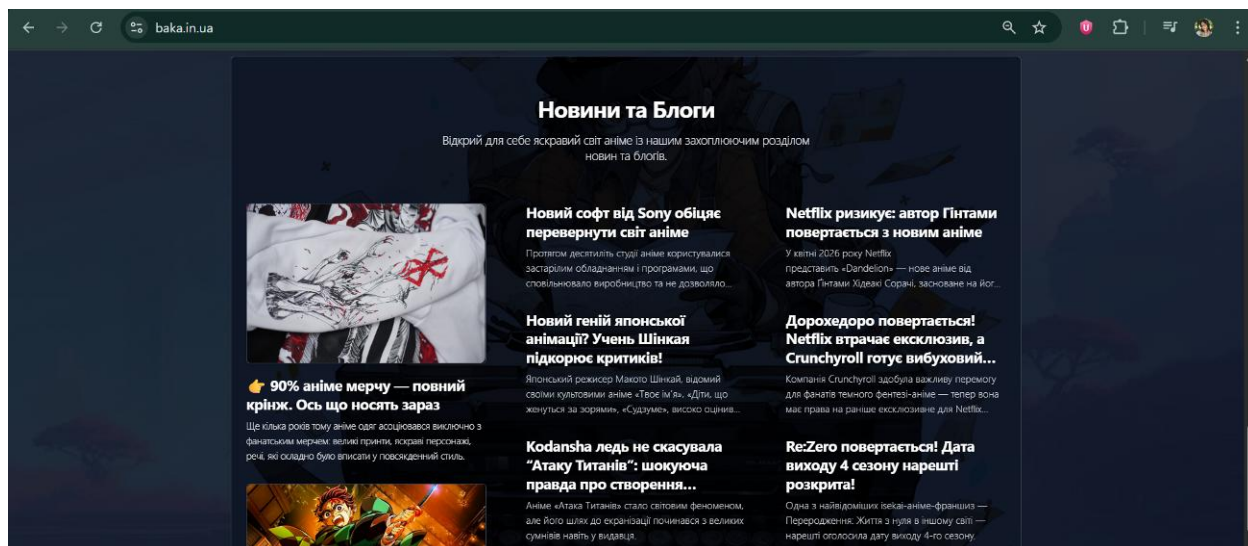


Рисунок 1.2 – Новини baka.in.ua

Наявна фільтрація за жанрами (див. рис. 1.3).

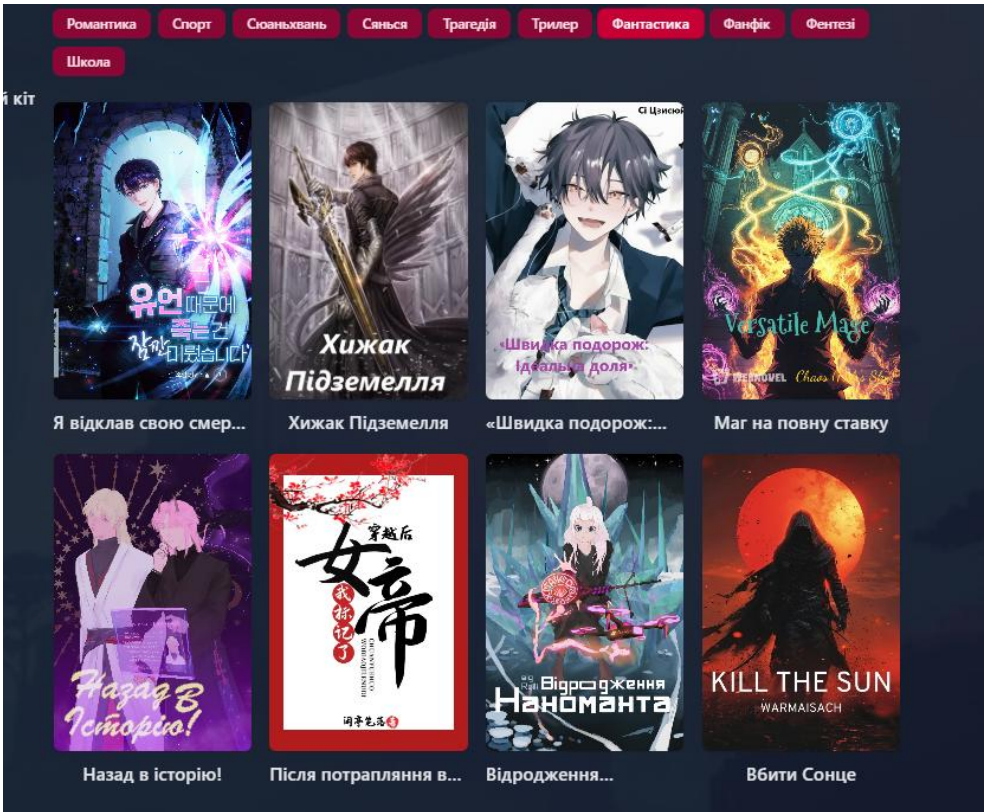


Рисунок 1.3 – Фільтрація за жанрами baka.in.ua

Кожний твір доступний для читання в перекладі українською (див. рис. 1.4). Сторінка містить загальну інформацію: кількість розділів, автора, тощо.

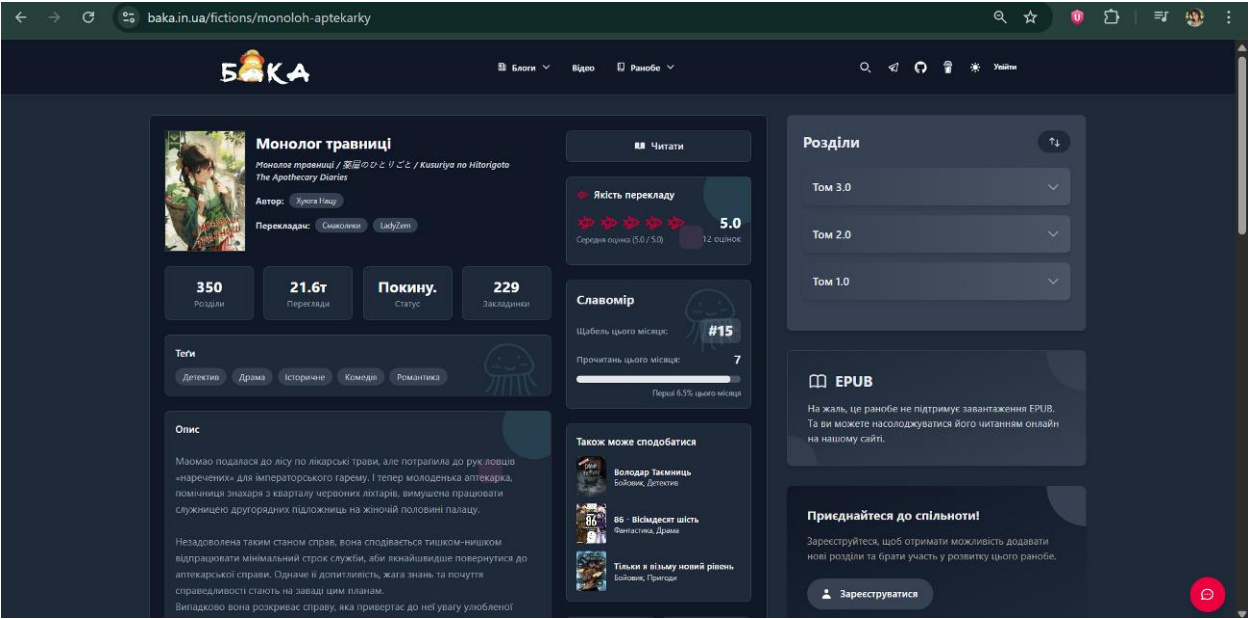


Рисунок 1.4 – Глава книги на baka.in.ua

Сторінка для читання книги виглядає приємно (див. рис. 1.5). У тексті можна змінювати шрифт та розмір. Як недолік зазначено відсутність змісту для зручного пересування розділами.

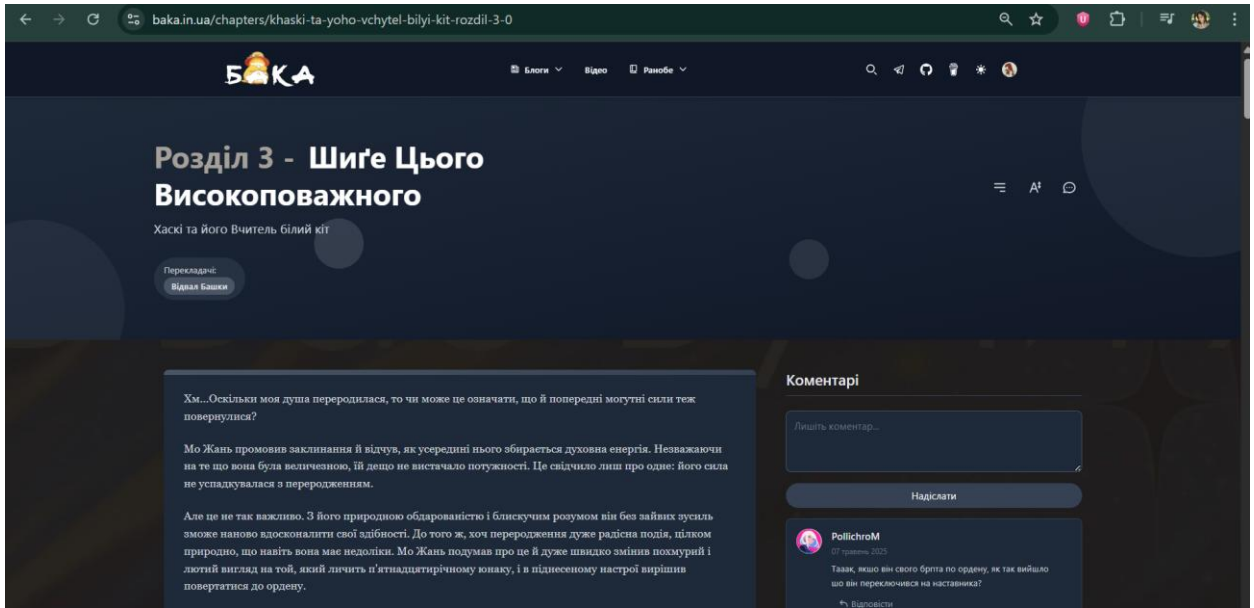


Рисунок 1.5 – Глава книги на baka.in.ua

Налаштування профілю прості: неможливо встановити власний аватар та статус (див. рис. 1.6), в профілі відображаються лише останні переглянуті книги (див. рис. 1.7).

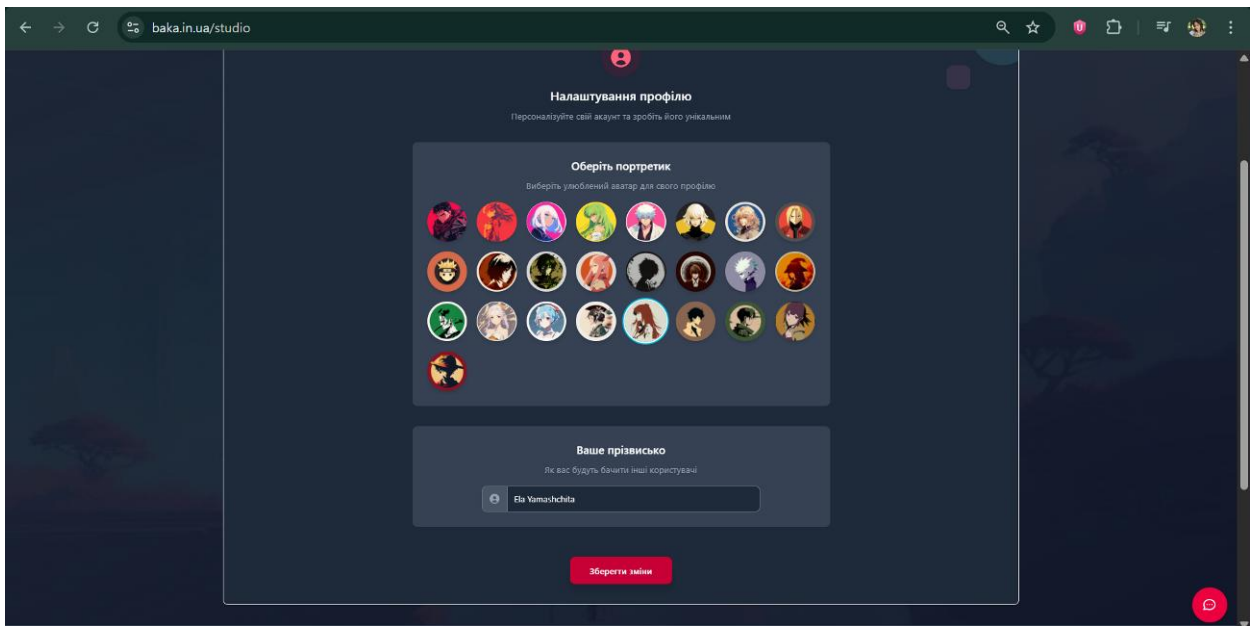


Рисунок 1.6 – Обмежена кількість аватарів на baka.in.ua

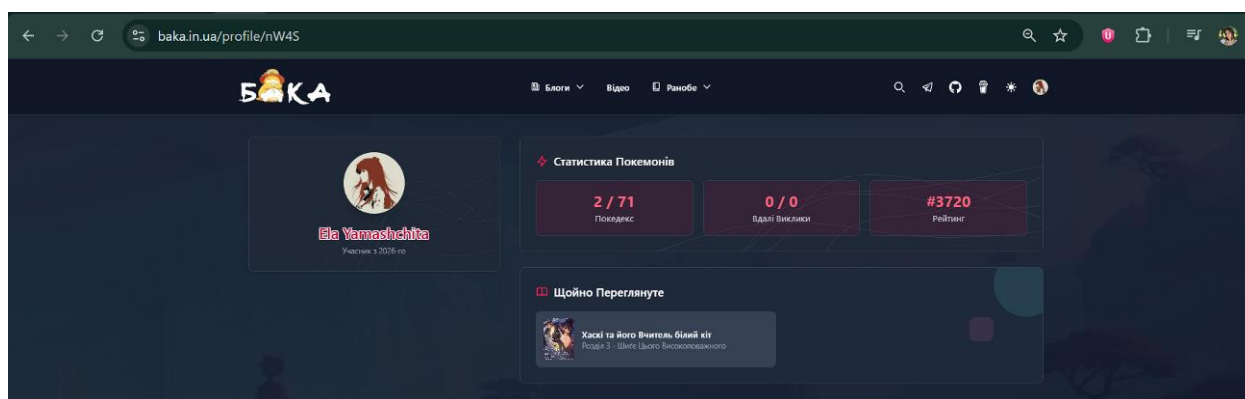


Рисунок 1.7 – Останні переглянуті книги на baka.in.ua

Загалом, цей вебсайт був єдиним знайденим повним вебзастосунком з заданої предметної області. Він зроблений якісно і може слугувати прикладом для подальшої роботи. Основний функціонал, схожий на baka.in.ua, також буде наявний у спроектованому застосунку, що продемонстровано у наступних пунктах. Із наявних переваг власного вебсайту хочеться відмітити більш широке редагування профілю: можливість встановити власний аватар, опис профілю, переглядати зміст, переходити до обраної глави, коментувати романи з наданням оцінки, встановлювати власний статус «Прочитано»/«В процесі»/«Заплановано» на книги, шукати авторів.

Отже, оскільки потреби української спільноти великі та скарг на замалу кількість контенту багато — має сенс створення застосунку, отримавши та проаналізувавши побажання користувачів.

1.2 Вимоги до програмного забезпечення

Слід визначити функціональні та нефункціональні вимоги (вимоги до інтерфейсу) для вебзастосунку.

Функціональні вимоги — основні функціональні можливості системи, тобто все, що відноситься до поведінки системи [5]. Вони мають

містити цілковий опис взаємодії програмного забезпечення та його користувачів, додатково зазначивши поведінку програмного забезпечення за різних умов:

- Система повинна дозволяти користувачу зареєструватися, створивши новий профіль.
- Система повинна дозволяти користувачу увійти з обраною роллю¹⁾.
- Система має надавати можливість користувачам з роллю «Користувач» оновлювати свій профіль: змінювати опис, ім'я користувача, аватар, тощо.
- Система має надавати можливість користувачам з роллю «Користувач» переглядати наявні книги.
- Система має надавати можливість користувачам з роллю «Користувач» переглядати профілі авторів.
- Система має надавати можливість користувачам з роллю «Користувач» шукати книги за назвою;
- Система має надавати можливість користувачам з роллю «Користувач» фільтрувати книги за жанром.
- Система має надавати можливість користувачам з роллю «Користувач» шукати авторів.
- Система має надавати можливість користувачам з роллю «Користувач» читати книги.
- Система має надавати можливість користувачам з роллю «Користувач» додавати книги до вподобань та оновлювати статус («Прочитано»/«В процесі»/«Заплановано»).
- Система має надавати можливість користувачам з роллю «Користувач» коментувати книги, відповідати іншим користувачам та оновлювати власні коментарі.

¹⁾Користувач, що є адміністратором в системі — може увійти з роллю «Адміністратор» або «Користувач» для, наприклад, тестування. Реєстрація можлива лише для користувачів з роллю «Користувач»

- Система має надавати користувачам з роллю «Адміністратор» можливості, надані користувачам з роллю «Користувач», окрім реєстрації.
- Система має надавати можливість користувачам з роллю «Адміністратор» видаляти будь-які коментарі.
- Система має надавати можливість користувачам з роллю «Адміністратор» додавати книги, видаляти та оновлювати про них інформацію.
- Система має надавати можливість користувачам з роллю «Адміністратор» додавати авторів, видаляти та оновлювати про них інформацію.

Далі розглянуто **нефункціональні вимоги**. Вони не несуть інформації про поведінку застосунку, зосереджуючись на обмеженнях системи та загальному зростанні якості [4]. До них входять вимоги до зручності, які є важливими, бо через важкість опанування сайту — попит на користування ним істотно знижується [10]. Отже, зазначено наступні вимоги до інтерфейсу [12]:

- Інтерфейс повинен бути інтуїтивним та візуально приємним²⁾;
- Інтерфейс має бути адаптивним під різні розміри екрану;
- Операції мають бути однозначними і не викликати непорозуміння у користувача;
- Поля для заповнення повинні мати підказки щодо заповнення;
- Користувач повинен мати зворотній зв'язок при успішному додаванні/оновленні/видаленні інформації, отриманні помилок;
- Інтерфейс повинен мати відповідні кнопки, які викликають передбачувані дії;
- Анімації та відгук інтерфейсу не має перевищувати 2-3 секунди;

²⁾Інтерфейс має бути не перевантаженим, мати гармонічну кольорову палітру, читабельний текст, плавні анімації.

1.3 Вибір засобів розробки

Оскільки вебзастосунки набирають популярність серед користувачів більше, ніж десктопні застосунки завдяки своїй зручності, компактності та відсутності потреби встановлення та оновлення [11], обрано наступну засоби розробки для вебдодатків:

- **HTML (HyperText Markup Language)** — це мова розмітки, що застосовується як основа для створення вебсторінок [3]. Вона визначає їх структуру та зовнішній вигляд.

- **CSS (Cascading Style Sheets)** — це мова стилів, яка дозволяє описати зовнішній вигляд документу та виокремити стилі в окремий .css файл [2], використовуючись разом з HTML. Завдяки цьому можливо зробити більш зручну, гнучку структуру та прописати складніші стилі (наприклад, додати анімації).

- **PostgreSQL** — об'єктно-реляційна система управління базами даних, корисна для великих проєктів. Є безкоштовною, має високу продуктивність та зручність. Опанована в попередній курсовій роботі, отже за наявності досвіду є кращим та стабільним варіантом.

- **React.js** — це JavaScript-бібліотека з відкритим кодом, яка є розширенням **JavaScript** (мова програмування, що є стандартом для створення інтерактивних вебсайтів), створена Facebook і використовується для ефективного керування елементами інтерфейсу користувача [22]. Її основний підхід базується на перевикористанні компонентів (незалежних частин інтерфейсу), які можна комбінувати між собою. Такий підхід забезпечує підвищення гнучкості розробки, зменшення дублювання коду та сприяє застосуванню принципів інкапсуляції. У свою чергу це полегшує тестування, супровід і подальше оновлення програмного забезпечення.

З бібліотек у клієнтській частині використано **ePub.js** (відповідає за рендеринг електронних книг у форматі EPUB [16]), **JWT Decode** (для

декодування JWT-токенів), **FontAwesome** (для іконок інтерфейсу, безкоштовна версія [8]).

З бібліотек у серверній частині використано **Express.js** (для обробки HTTP-запитів, маршрутизації та роботи з middleware [17]), **CORS** (для дозволу запитів між клієнтською та серверними частинами застосунку [14]), **PostgreSQL Client** (драйвер для підключення та взаємодії з базами даних [20]), **Dotenv** (для безпечного зберігання конфіденційних даних (паролі до бази, ключі Cloudinary) у файлі .env, щоб вони не потрапили у відкритий код [15]), **JSON Web Token** (для створення та перевірки токенів [18]), **Cookie Parser** (дозволяє серверу читати cookie-файли), **Cloudinary** (SDK для взаємодії з хмарним сховищем зображень [13]), **Multer** (middleware для обробки завантаження файлів [19]).

2 ПРОЄКТУВАННЯ СИСТЕМИ

Розроблюваний застосунок для читання книг реалізовано з використанням тришарової клієнт-серверної архітектури, що передбачає поділ системи на рівень представлення, рівень бізнес-логіки та рівень даних.

Клієнтська частина застосунку створена з використанням бібліотеки React і відповідає за взаємодію з користувачем, зокрема відображення списку книг, перегляд інформації про них та їх авторів, реалізацію пошуку, а також інтерфейс читання, можливість коментування, тощо. Взаємодія з сервером здійснюється шляхом надсилання HTTP-запитів і отримання відповідей.

Рівень бізнес-логіки реалізовано за допомогою серверу на Node.js, який обробляє запити клієнта, виконує перевірку та обробку даних, реалізує основні функції системи, а також забезпечує зв'язок із базою даних.

Рівень даних представлений базою даних у PostgreSQL, у якій зберігається інформація про користувачів, книги, авторів, жанри та коментарі.

Уся серверна частина реалізована як єдиний проєкт, що дає підстави віднести систему до **монолітної** архітектури, яка є доцільною для таких проєктів завдяки простоті розробки та розгортання.

Окрім цього, у застосунку використано архітектурний патерн **MVC (Model–View–Controller)**, який забезпечує розділення відповідальностей між компонентами системи: модель представлена класами сутностей, що описують структуру даних, контролери з сервісами реалізують обробку запитів і повернення результатів, а роль представлення виконує клієнтська частина на базі React.

2.1 ER-модель даних

Було розроблено базу даних, що включає такі сутності: «Користувачі», «Романи», «Жанри романів», «Письменники», «Коментарі».

Детальний опис сутностей наведено у таблиці 2.1.

Таблиця 2.1 – Опис сутностей та атрибутів

Сутність	Опис сутності	Атрибути сутності
Романи	Інформація про романи, кількість глав, статус завершення, тощо	– ID – Назва в оригіналі – Назва українською – Автор – Рік випуску – Статус завершення – Кількість томів – Кількість глав – Кількість екстр – Опис – Обкладинка – Текст книги
Жанри романів	Перелік доступних жанрів для романів	– ID – Жанр
Письменники	Перелік авторів романів та інформація про них	– ID – Повне ім'я – Дата народження
Користувачі	Інформація про кожного користувача, включаючи статус профіля за правами доступу (користувач/адміністратор)	– ID – Користувацьке ім'я – Розділ «про себе» – Логін – Пароль – Наявність статусу

		адміністратора – Зображення профілю
Коментарі	Інформація про кожний коментар під романами в інформаційній системі.	– ID – Роман, який коментуємо – ID коментаря, на який відповідаємо – Автор коментаря – Оцінка роману – Текст коментаря

Для графічного представлення обрана модель Гордона Евереста «вороняча лапка» (див. рис. 2.1)).

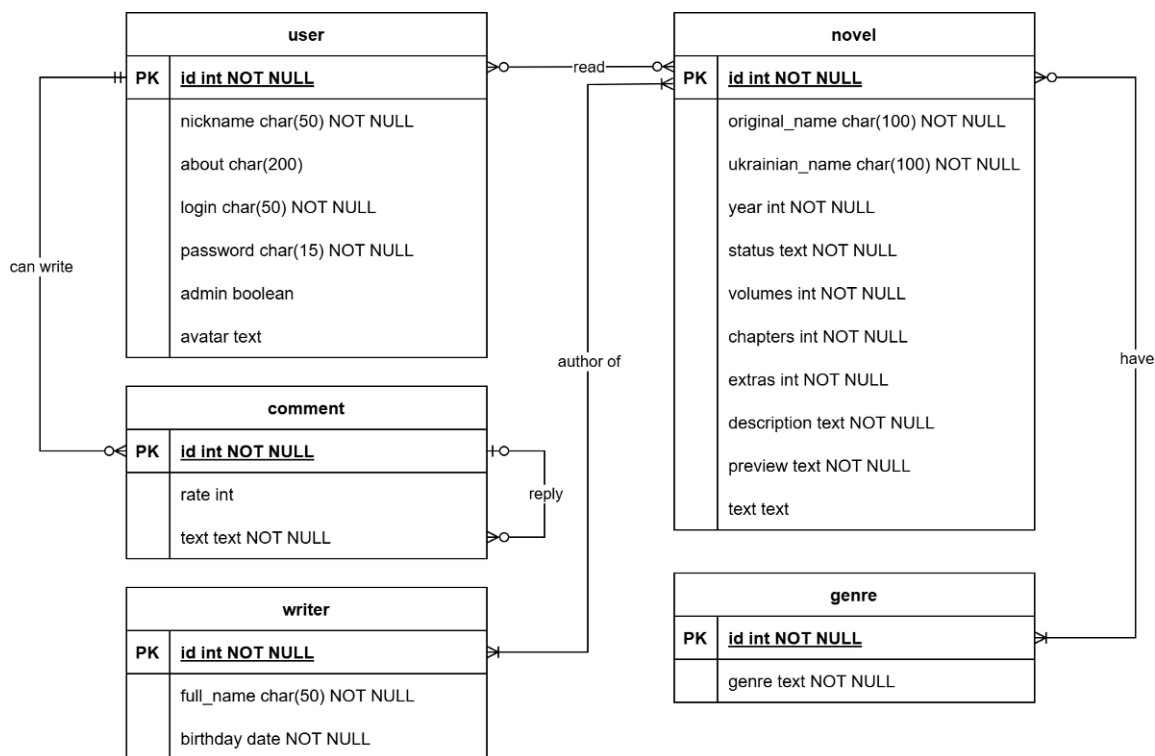


Рисунок 2.1 – Концептуальна модель сутностей та зв’язків моделі бази даних «Читальний зал»

Розглянемо зв'язки:

- сутність novels пов'язана з сутністю genres зв'язком «багато до багатьох» (зв'язок «have»), оскільки у одного роману обов'язково кілька жанрів, а кожний жанр зустрічається не обов'язково мінімум в одному романі. Зв'язок є обов'язковим зі сторони genres та необов'язковим зі сторони сутності novels;

- сутність writers пов'язана з сутністю novels зв'язком «багато до багатьох» (зв'язок «author of»), оскільки у одного роману обов'язково є мінімум один автор, а кожний письменник обов'язково є автором мінімум одного роману. Зв'язок є обов'язковим з обох сторін;

- сутність users пов'язана з сутністю comments зв'язком «один до багатьох» (зв'язок «can write»), оскільки один користувач не обов'язково може залишити мінімум один коментар, а кожен коментар обов'язково має тільки одного автора. Зв'язок є обов'язковим зі сторони users та необов'язковим зі сторони сутності comments;

- сутність comments пов'язана з сутністю comments зв'язком «один до багатьох» (зв'язок «reply»), оскільки кожний коментар не обов'язково є відповіддю на інший, а у одного коментаря необов'язково є мінімум один батьківський коментар. Зв'язок є не обов'язковим з двох сторін;

Проводимо створення ER-моделі. Додаються зовнішні ключі, зв'язки «багато до багатьох» перетворюються на проміжні таблиці, проводиться нормалізація:

- у жодному відношенні немає багатозначних атрибутів. Відношення знаходиться у 1НФ;

- кожен неключовий атрибут залежить від первинного ключа неприводимо. Це відповідає дійсності, бо всі первинні ключі є простими, отже залежність автоматично неприводима. Відношення знаходиться у 2НФ;

- кожен неключовий атрибут залежить від потенційного ключа. Це виконується, бо загалом всі неключові атрибути є характеристиками об'єктів сутності, на який вказують первинний(id)/потенційні ключі(повне ім'я).

Відношення знаходиться у 3НФ;

У результаті отримуємо готову ER-модель, яка доступна для реалізації у вигляді бази даних (див. рис. 2.2)).

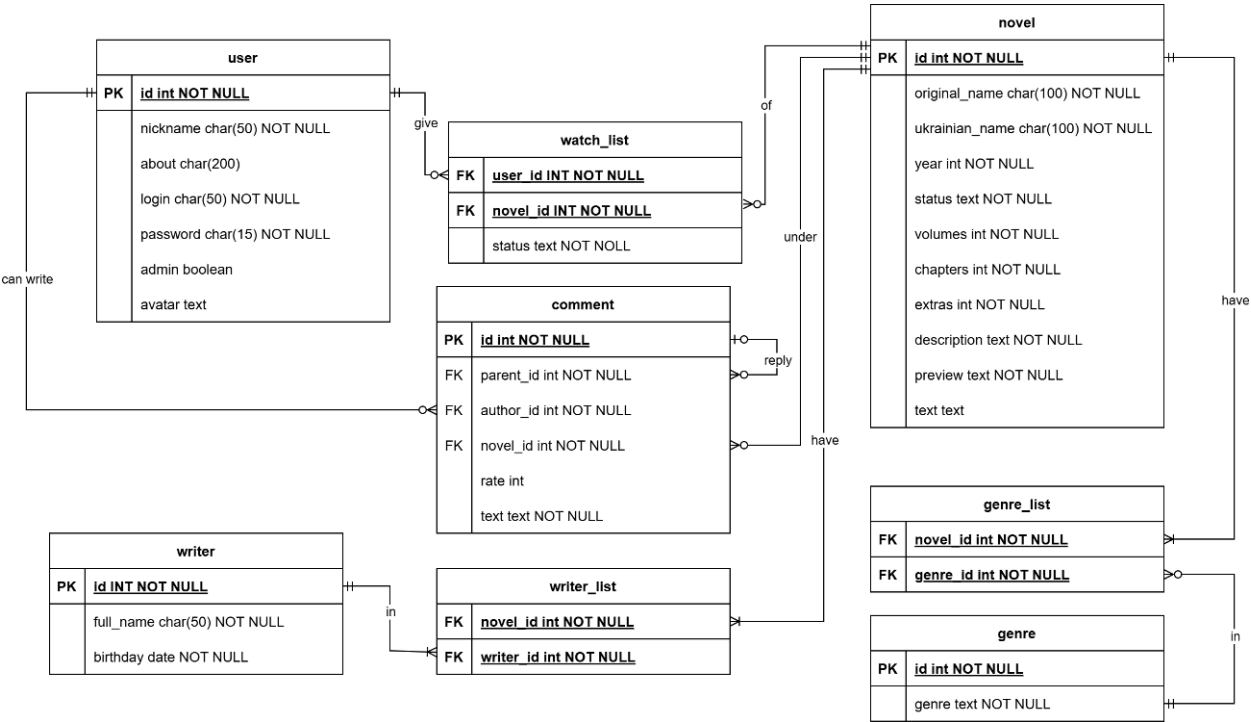


Рисунок 2.2 – ER-діаграма бази даних «Читальний зал»

2.2 Діаграма прецедентів

Наступним кроком розглянуто діаграму прецедентів, що демонструє доступний функціонал застосунку в залежності від обраної ролі користувача (див. рис. 2.3).

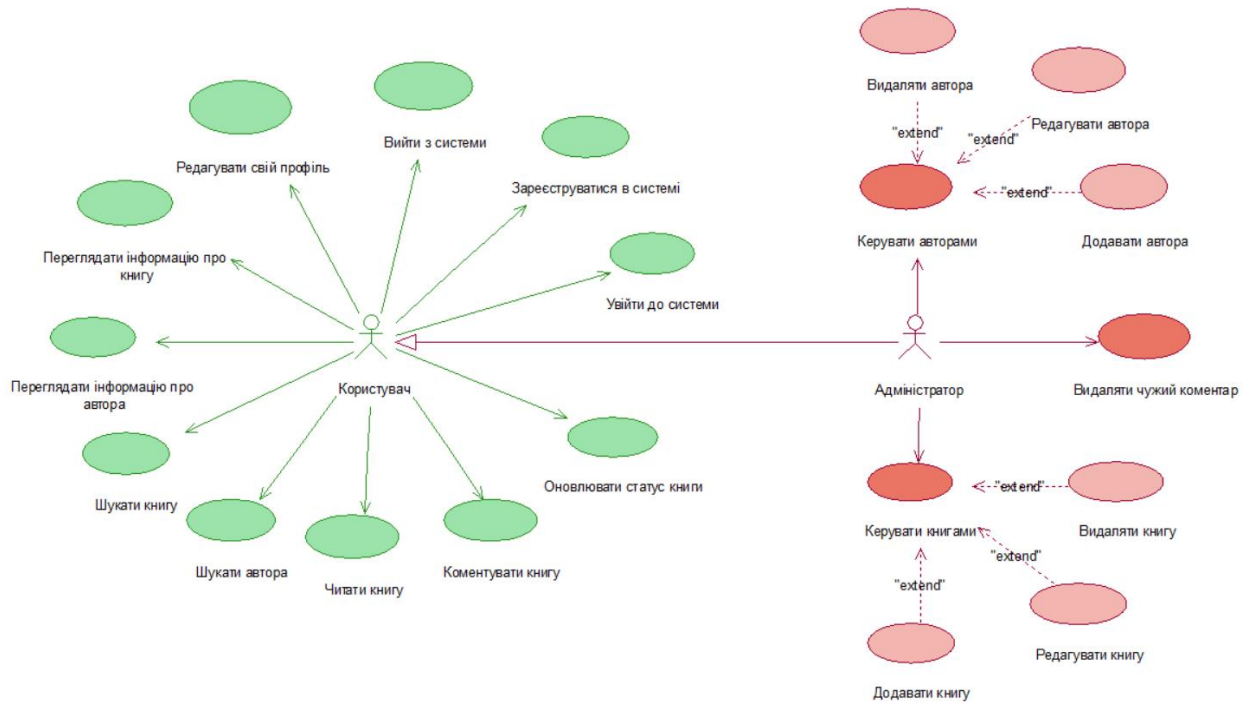


Рисунок 2.3 – Діаграма варіантів використання

Загалом існує 2 актори: звичайний «Користувач» і «Адміністратор». Перший має можливість переглядати всю доступну інформацію, коментувати її, видаляти свої коментарі та редагувати власний профіль, читати книги, шукати авторів та книги. «Адміністратор» є узагальненням «Користувача» з додатковими функціями: має всі ті ж самі можливості, але додатково може видаляти будь-які коментарі та може створювати, оновлювати та видаляти інформацію.

Розглянемо сценарії варіантів використання згідно діаграми:

1. Назва: Вхід до системи

Стислий опис: Цей варіант використання описує процес автентифікації «Користувача» або «Адміністратора» для отримання доступу до персоналізованих функцій системи (читання, коментування, керування контентом).

Основний сценарій:

Варіант використання починається, коли користувач натискає кнопку «Увійти».

- Система запитує логін (електронну пошту) та пароль, роль користувача.
- Користувач вводить облікові дані.
- Виконується підпорядкований сценарій «Перевірка автентичності».
- Після успішної перевірки відкривається доступ до головної сторінки, і варіант використання завершується.

Підпорядковані сценарії:

Підпорядкований сценарій «Перевірка автентичності»:

- Система зчитує введені дані та надсилає запит до бази даних.
- Система перевіряє наявність логіна та пароля в базі та наявну роль користувача.
- Якщо дані збігаються, сценарій завершується успішно.

Альтернативні сценарії подій:

- *Неправильний логін або пароль:* Якщо дані не збігаються, система виводить повідомлення про помилку. Користувач може спробувати увійти ще раз або перейти до варіанту використання «Зареєструватися у системі».

Передумови: Користувачу треба увійти до системи для роботи. Система перебуває у стані готовності для роботи.

Постумови: Користувач отримує доступ до свого облікового запису та доступних йому функцій.

2. Назва: Зареєструватися у системі

Стислий опис: Цей варіант використання описує процес створення нового облікового запису «Користувача». Після успішної реєстрації «Користувач» отримує доступ до функцій.

Основний сценарій:

Варіант використання починається, коли користувач натискає кнопку «Зареєструватися».

- Система запитує користувацьке ім'я, логін (електронну пошту) та пароль.
- Користувач вводить облікові дані.
- Виконується підпорядкований сценарій «Перевірка унікальності».
- Після успішної перевірки відкривається доступ до головної сторінки, і варіант використання завершується.

Підпорядковані сценарії:

Підпорядкований сценарій «Перевірка унікальності»:

- Система зчитує введені дані та надсилає запит до бази даних користувачів.
- Система перевіряє наявність логіна в базі.
- Якщо дані виявляються унікальними – сценарій завершується успішно.

Альтернативні сценарії подій:

- *Користувач із таким логіном вже існує:* Якщо під час виконання основного потоку виявиться, що введений логін вже зареєстрований в системі, то вона видає повідомлення про помилку. Користувач може змінити дані або перейти до варіанта використання «Увійти до системи», якщо він уже має акаунт.

Передумови: Перед початком виконання даного варіанта використання користувач не повинен бути авторизованим у системі.

Постумови: У системі створено новий профіль «Користувача». Дані збережені в базі даних, і користувач може використовувати їх для подальшої авторизації.

3. Назва: Вийти з системи

Стислий опис: Даний варіант використання описує процес завершення сеансу роботи «Користувача» або «Адміністратора» з системою.

Основний сценарій:

Варіант використання починається, коли користувач натискає кнопку «Вийти».

- Система запитує підтвердження завершення сеансу.
- Користувач підтверджує дію.
- Виконується підпорядкований сценарій «Анулювання сесії».
- Система перенаправляє користувача на початкову сторінку авторизації. Виконання варіанта використання завершується.

Підпорядковані сценарії:

Підпорядкований сценарій «Анулювання сесії»:

- Система видаляє токен доступу (Access Token) з локального сховища браузера.

Альтернативні сценарії подій:

- *Відміна виходу:* Якщо під час запиту на підтвердження користувач вирішує залишитися в системі та натискає «Скасувати», система повертає його до попереднього екрана. Стан сесії не змінюється.

Передумови: Перед початком виконання даного варіанта використання користувач повинен бути авторизованим у системі.

Постумови: Доступ до функцій закрито. Система більше не ідентифікує користувача, а всі поточні дані, що не були збережені, можуть бути втрачені.

4. Назва: Редагувати свій профіль

Стислий опис: Даний варіант використання дозволяє авторизованому користувачу змінити свої персональні дані (користувацьке ім'я, опис профілю), щоб підтримувати актуальність профілю в системі.

Основний сценарій:

Даний варіант використання починає виконуватися, коли користувач переходить у розділ «Налаштування профілю».

- Система завантажує та відображає поточні дані користувача у формі редагування.
- Користувач вносить зміни в одне або кілька полів.
- Користувач натискає кнопку «Зберегти зміни».
- Виконується підпорядкований сценарій «Збереження даних».
- Система виводить повідомлення про успішне оновлення профілю.

Підпорядковані сценарії:

Підпорядкований сценарій «Збереження даних»:

- Система надсилає оновлені дані на сервер.
- Після отримання успішної відповіді від серверу, система оновлює дані в поточному стані додатка, щоб зміни відобразилися миттєво без перезавантаження сторінки.

Альтернативні сценарії подій:

- *Скасування змін:* Користувач може в будь-який момент натиснути кнопку «Скасувати» або просто вийти з розділу налаштувань. У цьому випадку система не зберігає внесені зміни, і профіль залишається у попередньому стані.

5. Назва: Переглядати інформацію про книгу

Стислий опис: Даний варіант використання дозволяє авторизованому користувачу отримати детальну інформацію про вибрану книгу, включаючи опис, автора, рейтинг та відгуки.

Основний сценарій:

Даний варіант використання починає виконуватися, коли користувач натискає на назву або обкладинку книги в каталозі чи результатах пошуку.

- Система отримує унікальний ідентифікатор книги.

- Виконується підпорядкований сценарій «Завантаження даних про книгу».
- Система відображає сторінку книги, що містить:
 - Назву (в оригіналі та українською) та обкладинку;
 - Прізвище та ім'я автора (як посилання на його профіль);
 - Рік випуску;
 - Статус перекладу;
 - Оцінку;
 - Анотацію (короткий опис);
 - Жанри;
 - Кількість томів, глав, екстр;
 - Кнопку для читання (за наявності тексту книги);
 - Статус;
 - Кнопки редагування та видалення (якщо користувач в ролі «Адміністратор»);
 - Поточний рейтинг та список коментарів.

Підпорядковані сценарії:

Підпорядкований сценарій «Завантаження даних про книгу»:

- Система надсилає запит до бази даних для отримання повної інформації про книгу.
- Система підраховує середню оцінку на основі відгуків користувачів.

Альтернативні сценарії подій:

- *Книгу не знайдено:* Якщо під час завантаження даних виявиться, що книга була видалена або посилання некоректне, система виводить повідомлення про помилку.
- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

Передумови: Користувач повинен перебувати в розділі «Книги», або на головній сторінці, де відображаються останні новинки на сайті.

Постумови: Користувач отримав повну інформацію про видання. Користувач може перейти до виконання варіантів використання «Коментувати» або «Оновлювати статус».

6. Назва: Переглядати інформацію про автора

Стислий опис: Даний варіант використання дозволяє користувачу ознайомитися з інформацією про автора та переглянути список усіх його книг, що доступні в системі.

Основний сценарій:

Даний варіант використання починає виконуватися, коли користувач натискає на ім'я автора на сторінці книги, у результатах пошуку або в загальному списку авторів.

- Система отримує унікальний ідентифікатор автора.
- Виконується підпорядкований сценарій «Завантаження даних про автора».
- Система відображає сторінку автора, що містить:
 - Ім'я;
 - Дата народження;
 - Кнопки редагування та видалення (якщо користувач в ролі «Адміністратор»;
 - Список усіх книг даного автора, які зареєстровані в системі;

Підпорядковані сценарії:

Підпорядкований сценарій «Завантаження даних про автора»:

- Система надсилає запит до бази даних для отримання повної інформації про автора.

- Система виконує пошук у каталозі книг, щоб знайти всі записи, де цей автор вказаний як творець.

Альтернативні сценарії подій:

- *Автора не знайдено:* Якщо під час завантаження даних виявиться, що автора була видалено або посилання некоректне, система виводить повідомлення про помилку.

- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

Передумови: Користувач повинен перебувати в розділі «Автори», або на сторінці книги потрібного автора.

Постумови: Користувач отримав інформацію про автора та перелік його праць.

7. Назва: Коментувати

Стислий опис: Даний варіант використання дозволяє авторизованому користувачу залишати власні відгуки до книг, а також керувати вже створеними коментарями (редагувати або видаляти їх).

Основний сценарій:

Даний варіант використання починає виконуватися, коли користувач хоче залишити відгук на сторінці конкретної книги.

- Система відображає поле для введення тексту та кнопку «Надіслати» та наявні коментарі.

- Виконується один з підпорядкованих потоків («Додавати коментар», «Редагувати коментар», «Видаляти коментар», «Відповісти на коментар»).

- Система оновлює список коментарів на сторінці.

Підпорядковані сценарії:

Підпорядкований сценарій «Додавати коментар»:

- Користувач вводить текст коментаря.
- Користувач надає оцінку, заповнюючи потрібну кількість зірочок» (за бажанням).
- Користувач натискає кнопку «Надіслати».
- Система надсилає запит на сервер, додаючи до коментаря ідентифікатор користувача та книги.
- Після отримання успішної відповіді від серверу, система додає коментар у базу даних.
- За наявності оцінки в коментаря система оновлює рейтинг книги;

Підпорядкований сценарій «Редагувати коментар»:

- Користувач натискає кнопку з іконкою біля свого коментаря у розділі коментарів.
- Система перетворює текст коментаря на поле для редагування.
- Користувач вносить зміни та натискає «Зберегти».
- Система надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система оновлює текст коментаря.

Підпорядкований сценарій «Видаляти коментар»:

- Користувач натискає кнопку з іконкою кошику біля свого коментаря у розділі коментарів.
- Система запитує підтвердження видалення, попереджаючи про незворотність дії.
- Користувач підтверджує видалення.
- Система надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система видаляє коментар з бази даних з дочірніми коментарями за наявності та видаляє коментар зі списку на екрані.

Підпорядкований сценарій «Відповідати на коментар»:

- Користувач натискає кнопку «Відповісти» біля існуючого коментаря у розділі коментарів.
- Система відображає вкладену форму введення тексту безпосередньо під обраним коментарем.
- Користувач вводить текст відповіді.
- Система фіксує ідентифікатор батьківського коментаря.
- Користувач натискає кнопку «Надіслати».
- Виконується підпорядкований сценарій «Опублікувати коментар».
- Після успішного збереження система візуально відображає коментар із відступом (як вкладений елемент), щоб показати структуру діалогу.

Альтернативні сценарії подій:

- *Коментар не знайдено:* Якщо під час завантаження даних виявиться, що коментар було видалено або посилання некоректне, система виводить повідомлення про помилку.
- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.
- *Скасування редагування коментаря у сценарії «Редагувати коментар»:* Користувач може натиснути кнопку «Скасувати». У цьому випадку система не зберігає внесені зміни, і коментар залишиться у попередньому стані.
- *Скасування видалення коментаря у сценарії «Видаляти коментар»:* Користувач може натиснути кнопку «Скасувати». У цьому випадку система не зберігає внесені зміни, і коментарі залишаються у попередньому стані.

Передумови: Перед початком виконання даного варіанта використання користувач повинен перебувати на сторінці конкретної книги.

Постумови: Новий коментар або зміни в існуючому успішно збережені в базі даних. Користувачі, які переглядають цю ж книгу, побачать оновлений список коментарів.

8. Назва: Оновити статус книги

Стислий опис: Даний варіант використання дозволяє авторизованому користувачу керувати своїм списком літератури, позначаючи книги відповідно до етапу читання («Заплановано», «В процесі», «Прочитано»).

Основний сценарій:

Даний варіант використання починає виконуватися, коли користувач натискає на вибір статусу на сторінці книги.

- Система пропонує вибір статусу для оновлення: «Заплановано», «В процесі», «Прочитано» в якості випадаючого списку.
- Користувач обирає один із запропонованих варіантів статусу («Заплановано», «В процесі», «Прочитано»).
- Виконується підпорядкований сценарій «Збереження статусу».
- Система оновлює візуальне відображення кнопки.
- Система виводить коротке повідомлення про успішне оновлення.

Підпорядковані сценарії:

Підпорядкований сценарій: «Збереження статусу»:

- Якщо книга нова для користувача, система створює новий запис у базі даних із вказаним статусом (якщо книга вже була в списку, система оновлює існуюче значення статусу на нове).
- Система оновлює дані в поточному стані додатка, щоб зміни відобразилися миттєво без перезавантаження сторінки.

Альтернативні сценарії подій:

- *Помилка синхронізації:* Якщо під час збереження статусу виникла помилка мережі, система виводить повідомлення про помилку. Поточний статус книги на екрані повертається до попереднього значення.

Передумови: Перед початком виконання даного варіанта використання користувач повинен перебувати на сторінці конкретної книги.

Постумови: Обраний статус книги успішно збережений. Книга автоматично з'являється у відповідній категорії в профілі користувача.

9. Назва: Шукати книгу

Стислий опис: Даний варіант використання дозволяє користувачу знайти конкретну книгу в загальному каталозі, використовуючи текстовий пошук за назвою або вибір конкретного жанру.

Основний сценарій:

Даний варіант використання починає виконуватися, коли користувач переходить до розділу «Книги».

- Система відображає поле для текстового введення та список доступних жанрів.
- Користувач обирає спосіб пошуку (за назвою або жанром) і виконується один з підпорядкованих потоків.
- Система виводить список книг, що відповідають критеріям.

Підпорядковані сценарії:

Підпорядкований сценарій: «Пошук за назвою»

- Користувач вводить текст у поле пошуку.
- Користувач натискає кнопку «Пошук».
- Система зчитує введений рядок символів.
- Система виконує фільтрацію бази даних за частковим збігом.

Підпорядкований сценарій: «Пошук за жанром»:

- Користувач натискає обраний жанр з переліку.
- Система формує запит, де головним критерієм є ідентифікатор жанру.
- Система виконує фільтрацію бази даних за жанром.

Альтернативні сценарії подій:

- *Помилка мережі:* Якщо під час пошуку виникла помилка мережі, система виводить повідомлення про помилку.
- *Користувач не задав жодного параметру:* Система відображає всі наявні книги.

Передумови: Користувач відкрив сторінку «Книги». Система має стабільне з'єднання з сервером бази даних.

Постумови: Користувач бачить набір книг, що відповідають його запиту.

10. Назва: Шукати автора

Стислий опис: Даний варіант використання дозволяє користувачу знайти конкретного автора в загальному каталозі, використовуючи текстовий пошук за іменем.

Основний сценарій:

Даний варіант використання починає виконуватися, коли користувач переходить до розділу «Автори».

- Система відображає поле для текстового введення.
- Виконується підпорядкований сценарій «Обробка пошукового запиту».
- Система виводить список авторів, що були знайдені в результаті пошуку.

Підпорядковані сценарії:

Підпорядкований сценарій: «Обробка пошукового запиту»

- Користувач вводить текст у поле пошуку.
- Користувач натискає «Пошук».
- Система зчитує введенний рядок символів.
- Система виконує фільтрацію бази даних за частковим збігом.

Альтернативні сценарії подій:

- *Помилка мережі:* Якщо під час пошуку виникла помилка мережі, система виводить повідомлення про помилку.
- *Користувач не задав жодного параметру:* Система відображає всіх наявні авторів.

Передумови: Користувач відкрив сторінку «Автори». Система має стабільне з'єднання з сервером бази даних.

Постумови: Користувач бачить список авторів, що відповідають його запиту.

11.Назва: Читати книгу

Стислий опис: Даний варіант використання дозволяє користувачу переглядати текст книги, налаштовувати зручний візуальний стиль (тему, шрифт, розмір) та зберігати поточну позицію або важливі фрагменти за допомогою закладок.

Основний сценарій:

Даний варіант використання починається, коли користувач обирає конкретну книгу та натискає «Читати» на її сторінці.

- Система завантажує першу главу книги та відображає її на екрані.
- Система відображає верхнє меню «Параметри тексту» (кнопка зміни теми, кнопка зміни шрифту, кнопка зміни розміру шрифту, кнопки «наступна глава»/«попередня глава») та нижнє меню «Зміст та закладки» (кнопка з іконкою змісту, кнопка з іконкою закладки, кнопки «наступна глава»/«попередня глава»).
- Виконується підпорядкований сценарій «Налаштування тексту» (за бажанням користувача).
- Виконується один з підпорядкованих потоків («Додавати закладку», «Переходити до закладки», «Видаляти закладку» за бажанням користувача).

- Виконується підпорядкований сценарій «Перейти до іншої глави» (за бажанням користувача).
- Система автоматично зберігає останню прочитану сторінку при закритті книги.

Підпорядковані сценарії:

Підпорядкований сценарій «Налаштування тексту»

- Користувач натискає на кнопку зміни теми у верхньому меню «Параметри тексту».
- Система застосовує до тексту кольорову схему, наступну зі списку з трьох наявних можливих тем («Дракон» (фірмова тема проекту), світла, темна).
- Користувач натискає на кнопку зміни шрифту у верхньому меню «Параметри тексту».
- Система застосовує до тексту шрифт, наступний зі списку з трьох наявних можливих шрифтів («Memory» (фірмовий стиль проекту), «Times New Roman», «Arial»).
- Користувач натискає на кнопку зміни розміру шрифту у верхньому меню «Параметри тексту» («+»/«-»).
- Система змінює розмір тексту («збільшує»/«зменшує»).

Підпорядкований сценарій: «Додавати закладку»

- Користувач натискає на абзац тексту, який бажає відмітити.
- Система відображає кнопку з іконкою закладки біля обраного абзацу тексту.
- Користувач натискає на кнопку.
- Система фіксує ідентифікатор глави, абзацу, короткий зміст абзацу та дату створення.
- Система стилізує обраний абзац за допомогою кольору.
- Система оновлює список «Мої закладки», додаючи нову закладку.

Підпорядкований сценарій: «Переходити до закладки»

- Користувач натискає на кнопку з іконкою закладки у нижньому меню «Зміст та закладки».
- Система виводить список закладок.
- Користувач натискає на обрану закладку у списку.
- Система оновлює сторінку.
- Користувач опиняється на місці з виділеним фрагментом, який підсвічується.

Підпорядкований сценарій: «Видаляти закладку»

- Користувач натискає на кнопку з іконкою закладки у нижньому меню «Зміст та закладки».
- Користувач обирає потрібний фрагмент та натискає кнопку «Видалити» біля закладки, або виконується сценарій «Переходити до закладки» і там користувач натискає на іконку закладки.
- Система видаляє з середовища закладку, а фрагмент більше не виділяється за допомогою стилів.

Підпорядкований сценарій «Перейти до наступної/попередньої глави»

- Користувач натискає на кнопку «наступна глава»/«попередня глава» у верхньому/нижньому меню «Зміст та закладки».
- Система відображає наступну/попередню главу.

Підпорядкований сценарій «Перейти до обраної глави»

- Користувач натискає на кнопку з іконкою змісту у нижньому меню «Зміст та закладки».
- Система виводить зміст книги у вигляді списку.
- Користувач обирає зі списку главу.
- Система відображає обрану главу.

Альтернативні сценарії подій:

- *Помилка мережі:* Якщо під час завантаження глави виникла помилка мережі, система виводить повідомлення про помилку.

– *Початкова позиція:* Якщо користувач попередньо зупинився на іншій главі, то при наступному поверненні до книги, першою відобразиться глава зі збереженої раніше позиції.

Передумови: Користувач авторизований у системі та натиснув кнопку «Читати» на сторінці обраної книги.

Постумови: Система зберігає обрані налаштування відображення та створені закладки для цієї книги.

12. Назва: Додавати книгу

Стислий опис: Даний варіант використання дозволяє «Адміністратору» додати нову книгу до системи.

Основний сценарій:

Даний варіант використання починає виконуватися, коли «Адміністратор» знаходиться на сторінці «Книги».

- Система відображає всі наявні книги.
- «Адміністратор» натискає кнопку з іконкою плюса.
- Система відображає порожню форму для заповнення.
- «Адміністратор» вводить дані та натискає «Зберегти».
- Виконується підпорядкований сценарій «Перевірка даних».

Підпорядкований сценарій «Перевірка даних»:

- Система перевіряє коректність заповнення полів та надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система додає книгу до бази даних та виводить повідомлення про успішну операцію.

Альтернативні сценарії подій:

- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

Передумови: Перед початком виконання даного варіанта використання «Адміністратор» повинен бути авторизований у системі з відповідними правами доступу.

Постумови: Нову книгу додано до системи. Внесені зміни стають доступними для перегляду всім користувачам системи.

13. Назва: Редагувати книгу

Стислий опис: Даний варіант використання дозволяє «Адміністратору» редагувати інформацію про обрану книгу.

Основний сценарій:

Даний варіант використання починає виконуватися, коли «Адміністратор» знаходиться на сторінці книги.

- Система відображає інформацію про обрану книгу.
- «Адміністратор» натискає кнопку з іконкою олівця.
- Система відображає заповнену форму з даними про книгу.
- «Адміністратор» вносить зміни та натискає «Зберегти».
- Виконується підпорядкований сценарій «Перевірка даних».

Підпорядкований сценарій «Перевірка даних»:

- Система перевіряє коректність заповнення полів та надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система оновлює книгу у базі даних та виводить повідомлення про успішну операцію.

Альтернативні сценарії подій:

- *Книгу не знайдено:* Якщо під час завантаження даних виявиться, що книгу було видалено або посилання некоректне, система виводить повідомлення про помилку.
- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

Передумови: Перед початком виконання даного варіанта використання «Адміністратор» повинен бути авторизований у системі з відповідними правами доступу.

Постумови: Книгу оновлено у системі. Внесені зміни стають доступними для перегляду всім користувачам системи.

14. Назва: Видаляти книгу

Стислий опис: Даний варіант використання дозволяє «Адміністратору» видаляти обрану книгу.

Основний сценарій:

Даний варіант використання починає виконуватися, коли «Адміністратор» знаходиться на сторінці книги.

- Система відображає інформацію про обрану книгу.
- «Адміністратор» натискає кнопку з іконкою кошику.
- Система запитує підтвердження видалення.
- «Адміністратор» підтверджує видалення.
- Виконується підпорядкований сценарій «Видалення з системи».

Підпорядкований сценарій «Видалення з системи»:

- Система надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система видаляє книгу з бази даних та виводить повідомлення про успішну операцію.

Альтернативні сценарії подій:

- *Книгу не знайдено:* Якщо під час завантаження даних виявиться, що книгу було видалено або посилання некоректне, система виводить повідомлення про помилку.
- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

– *Скасування видалення книги:* «Адміністратор» може натиснути кнопку «Скасувати». У цьому випадку система не зберігає внесені зміни, і книги залишаються у попередньому стані.

Передумови: Перед початком виконання даного варіанта використання «Адміністратор» повинен бути авторизований у системі з відповідними правами доступу.

Постумови: Книгу видалено з системи. Внесені зміни стають доступними для перегляду всім користувачам системи.

15. Назва: Додавати автора

Стислий опис: Даний варіант використання дозволяє «Адміністратору» додати нового автора до системи.

Основний сценарій:

Даний варіант використання починає виконуватися, коли «Адміністратор» знаходиться на сторінці «Автори».

- Система відображає всіх наявних авторів.
- «Адміністратор» натискає кнопку з іконкою плюса.
- Система відображає порожню форму для заповнення.
- «Адміністратор» вводить дані та натискає «Зберегти».
- Виконується підпорядкований сценарій «Перевірка даних».

Підпорядкований сценарій «Перевірка даних»:

- Система перевіряє коректність заповнення полів та надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система додає автора до бази даних та виводить повідомлення про успішну операцію.

Альтернативні сценарії подій:

- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

Передумови: Перед початком виконання даного варіанта використання «Адміністратор» повинен бути авторизований у системі з відповідними правами доступу.

Постумови: Нового автора додано до системи. Внесені зміни стають доступними для перегляду всім користувачам системи.

16. Назва: Редагувати автора

Стислий опис: Даний варіант використання дозволяє «Адміністратору» редагувати інформацію про обраного автора.

Основний сценарій:

Даний варіант використання починає виконуватися, коли «Адміністратор» знаходиться на сторінці автора.

- Система відображає інформацію про обрану книгу.
- «Адміністратор» натискає кнопку з іконкою олівця.
- Система відображає заповнену форму з даними про автора.
- «Адміністратор» вносить зміни та натискає «Зберегти».
- Виконується підпорядкований сценарій «Перевірка даних».

Підпорядкований сценарій «Перевірка даних»:

- Система перевіряє коректність заповнення полів та надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система оновлює автора у базі даних та виводить повідомлення про успішну операцію.

Альтернативні сценарії подій:

- *Автора не знайдено:* Якщо під час завантаження даних виявиться, що автора було видалено або посилання некоректне, система виводить повідомлення про помилку.

– *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

Передумови: Перед початком виконання даного варіанта використання «Адміністратор» повинен бути авторизований у системі з відповідними правами доступу.

Постумови: Автора оновлено у системі. Внесені зміни стають доступними для перегляду всім користувачам системи.

17. Назва: Видаляти автора

Стислий опис: Даний варіант використання дозволяє «Адміністратору» видаляти обраного автора.

Основний сценарій:

Даний варіант використання починає виконуватися, коли «Адміністратор» знаходиться на сторінці автора.

- Система відображає інформацію про обраного автора.
- «Адміністратор» натискає кнопку з іконкою кошику.
- Система запитує підтвердження видалення.
- «Адміністратор» підтверджує видалення.
- Виконується підпорядкований сценарій «Видалення з системи».

Підпорядкований сценарій «Видалення з системи»:

- Система надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система видаляє автора з бази даних та виводить повідомлення про успішну операцію.

Альтернативні сценарії подій:

- *Автора не знайдено:* Якщо під час завантаження даних виявиться, що автора було видалено або посилання некоректне, система виводить повідомлення про помилку.

– *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.

– *Скасування видалення книги:* «Адміністратор» може натиснути кнопку «Скасувати». У цьому випадку система не зберігає внесені зміни, і автори залишаються у попередньому стані.

Передумови: Перед початком виконання даного варіанта використання «Адміністратор» повинен бути авторизований у системі з відповідними правами доступу.

Постумови: Автора видалено з системи. Внесені зміни стають доступними для перегляду всім користувачам системи.

18. Назва: Видаляти чужий коментар

Стислий опис: Даний варіант використання дозволяє адміністратору видаляти коментар будь-якого користувача у разі порушення ними правил спільноти (спам, нецензурна лексика, образи тощо) для підтримки порядку в системі.

Основний сценарій:

Даний варіант використання починає виконуватися, коли адміністратор, переглядаючи сторінку книги, виявляє коментар, що підлягає видаленню.

– «Адміністратор» натискає кнопку з іконкою кошика біля коментаря.

– Система запитує підтвердження видалення, попереджаючи про незворотність дії.

– «Адміністратор» підтверджує видалення.

– Виконується підпорядкований сценарій «Видаляти коментар з бази даних».

– Система оновлює список коментарів на сторінці.

Підпорядковані сценарії:

Підпорядкований сценарій «Видаляти коментар з бази даних»:

- Система надсилає запит на сервер.
- Після отримання успішної відповіді від серверу, система видаляє коментар з бази даних з дочірніми коментарями за наявності

Альтернативні сценарії подій:

- *Коментар не знайдено:* Якщо під час завантаження даних виявиться, що коментар було видалено або посилання некоректне, система виводить повідомлення про помилку.
- *Помилка завантаження контенту:* Якщо сервер не відповідає або виникла помилка мережі, система відображає повідомлення про технічну помилку.
- *Скасування видалення коментаря у сценарії «Видаляти коментар»:* «Адміністратор» може натиснути кнопку «Скасувати». У цьому випадку система не зберігає внесені зміни, і коментарі залишаються у попередньому стані.

Передумови: Перед початком виконання даного варіанта використання «Адміністратор» повинен перебувати на сторінці конкретної книги.

Постумови: Обраний чужий коментар видалено з бази даних та він більше не відображається для жодного користувача системи.

2.3 Аналіз прецедентів за допомогою діаграм послідовностей та кооперацій

Наступним кроком розглянуто діаграму послідовностей, отриману в результаті аналізу сценарію «Читати книгу» з попереднього розділу в якості основного варіанту використання (див. рис. 2.4).

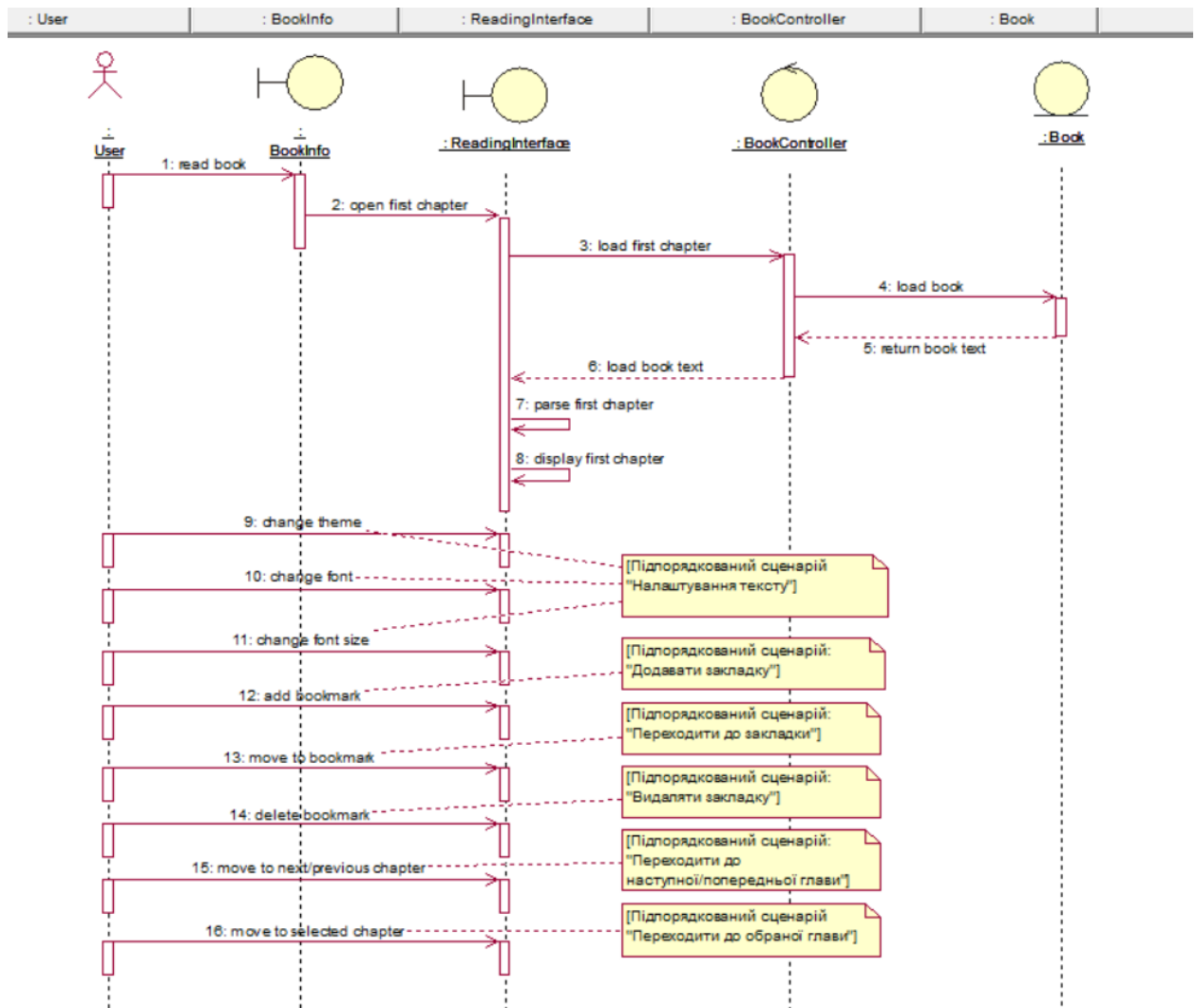


Рисунок 2.4 – Діаграма послідовностей

Наявні класи:

- **BookInfo** – boundary-клас для відображення інформації про книгу, з яким напряму взаємодіє користувач, натискаючи кнопку «Читати книгу».
- **ReadingInterface** – boundary-клас для відображення тексту книги, що дозволяє її читати. Клас дозволяє отримати першу главу книги та відобразити її (вважаємо за початковий стан відсутність попереднього збереження поточної позиції користувача відносно глав). Користувач також може взаємодіяти з класом, щоб налаштувати текст, керувати закладками, тощо.

- **BookController** – controller-клас для взаємодії ReadingInterface та Book. Він відповідає за основну логіку отримання тексту книги.
- **Book** – entity-клас, що відображає модель книги. Дозволяє отримати текст.

З отриманої діаграми створюємо діаграму кооперацій (див. рис. 2.5).

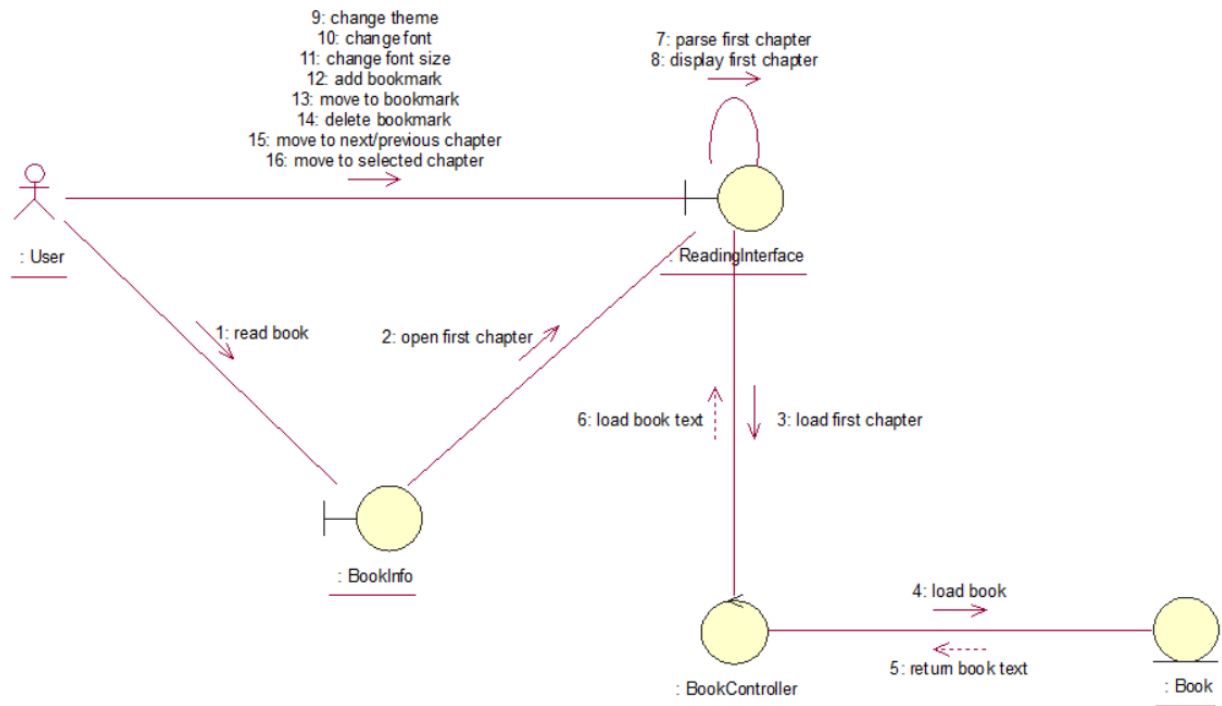


Рисунок 2.5 – Діаграма кооперацій

2.4 Діаграма класів

Діаграма класів представляє собою опис зав'язків об'єктів застосунку між собою, відображаючи взаємодію клієнтських та серверних частин (див. рис. 2.6). *Червоним* представлені класи, що відповідають за функціонал, пов'язаний з **авторами** книг, *зеленим* — з **книгами** та їх складовими (**жанри**, **коментарі** відмічені іншим відтінком *зеленого* кольору), *лавандовим* — з **користувачем** системи. *Бірюзовим* також висвітлено додаткові **коментарі** до певних сторінок, елементів, тощо.

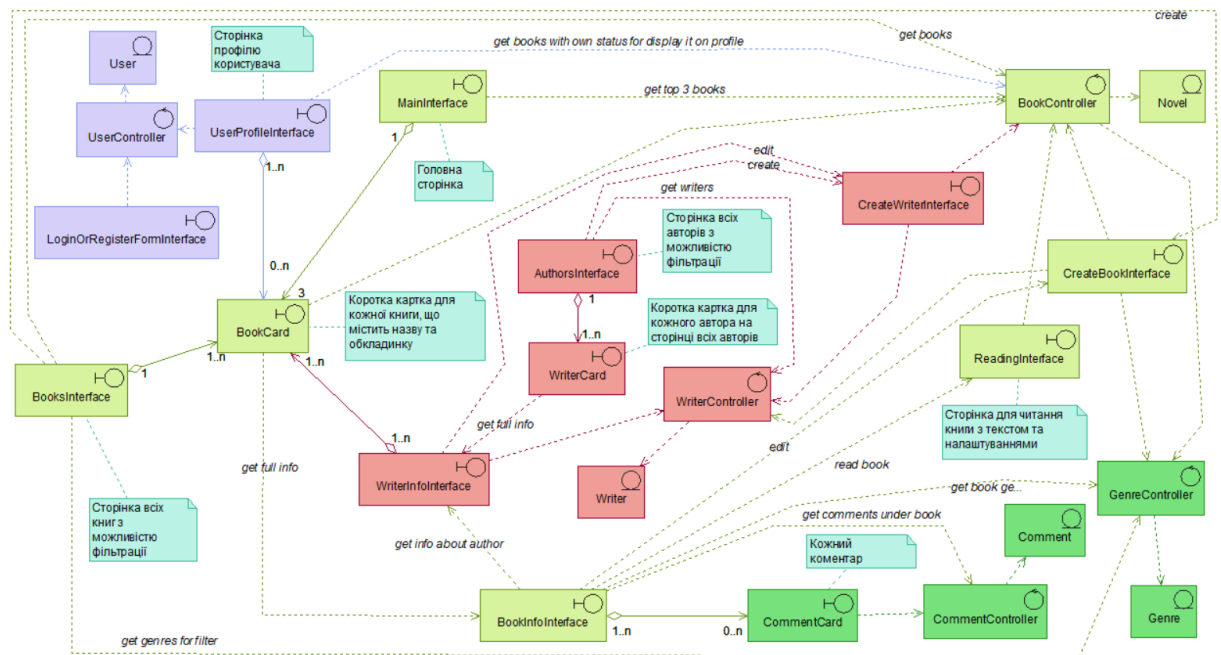


Рисунок 2.6 – Загальна діаграма класів

З міркувань наочності, загальну діаграму розбито кольорами на групи, як зазначалося вище. Отже, має сенс далі розглянути кожну групу окремо, опускаючи зв'язки між самими групами.

Діаграма класів групи «Автори» відображає методи, атрибути та додаткову специфікацію (див. рис. 2.7).

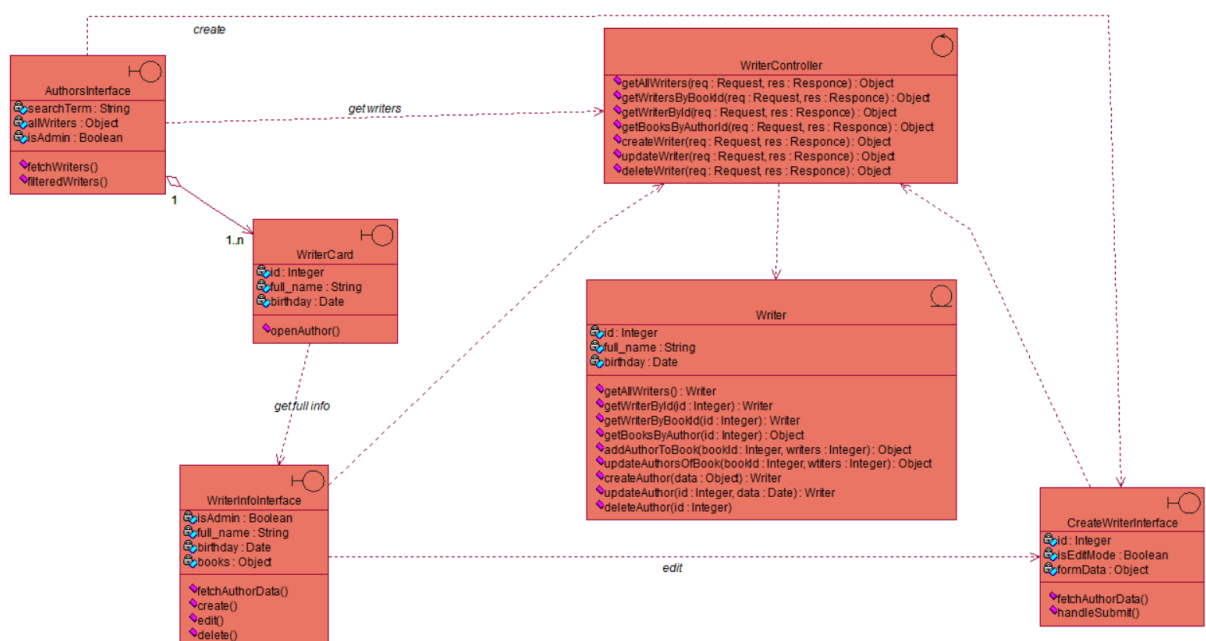


Рисунок 2.7 – Діаграма класів «Автори»

Діаграма класів групи «Книги» відображає методи, атрибути, але не відображає деяку специфікацію, зокрема типи даних (див. рис. 2.8). Таке рішення було прийняте через проблему читабельності при більшій деталізації діаграми.

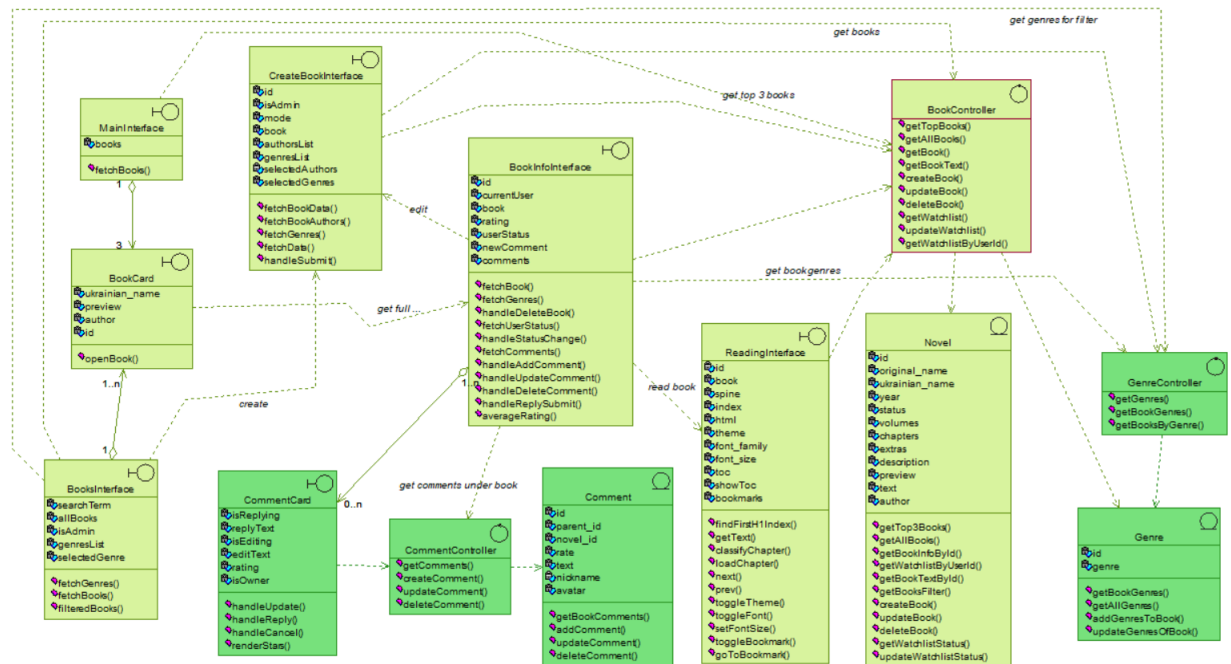


Рисунок 2.8 – Діаграма класів «Книги»

Діаграма класів групи «Користувачі» відображає методи, атрибути та додаткову специфікацію (див. рис. 2.9).

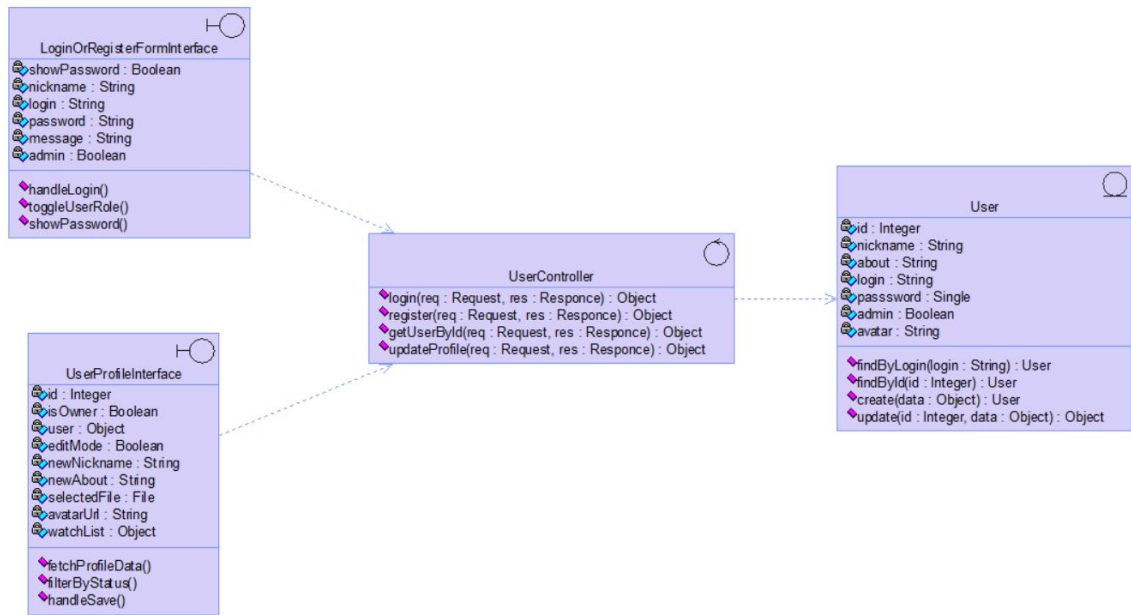


Рисунок 2.9 – Діаграма класів «Користувачі»

2.5 Діаграма діяльності

Розглянемо діаграму діяльності, що відображає основний сценарій «Читати книгу». Основна діаграма є верхньорівневим представленням (див. рис. 2.10). Дії «Register/Login» та «Open book for reading» мають у замітках посилання на окремі діаграми, присвячені їм (див. рис. 2.11 та рис. 2.12 відповідно).

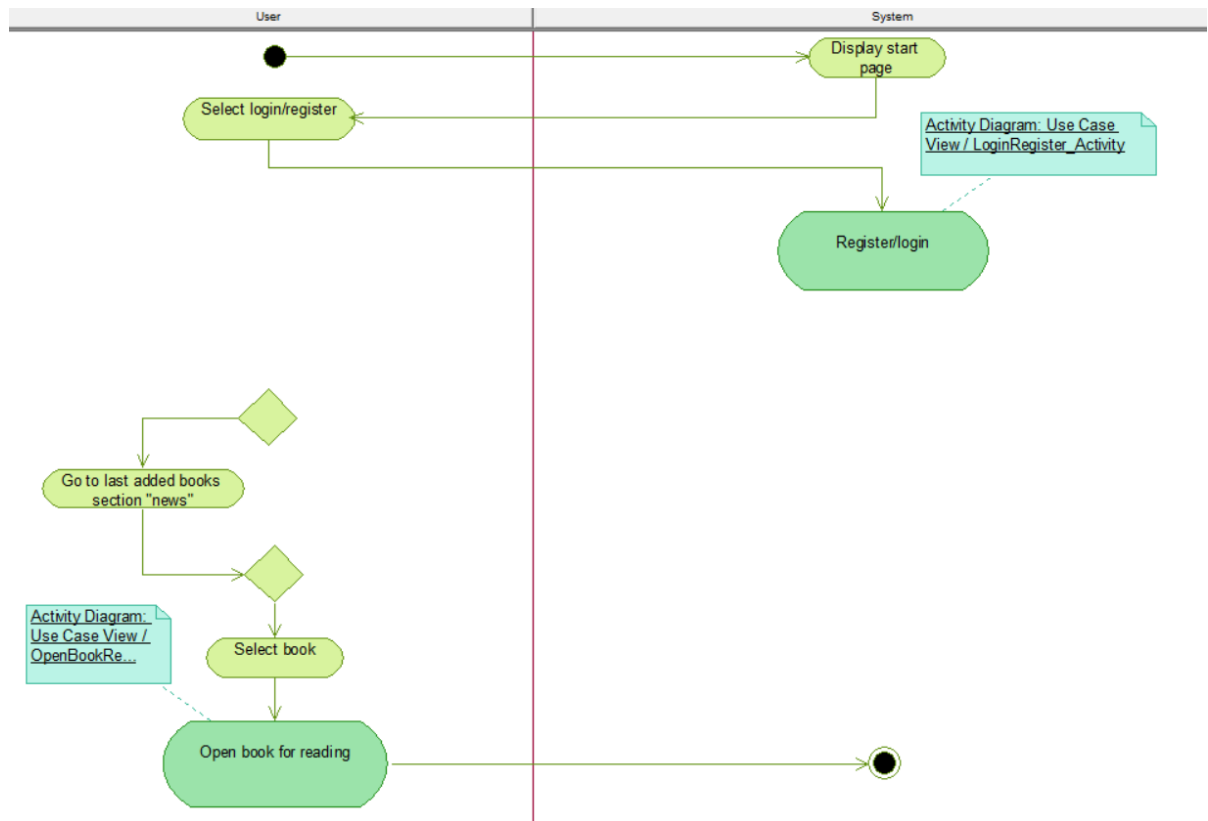


Рисунок 2.10 – Загальна діаграма діяльності

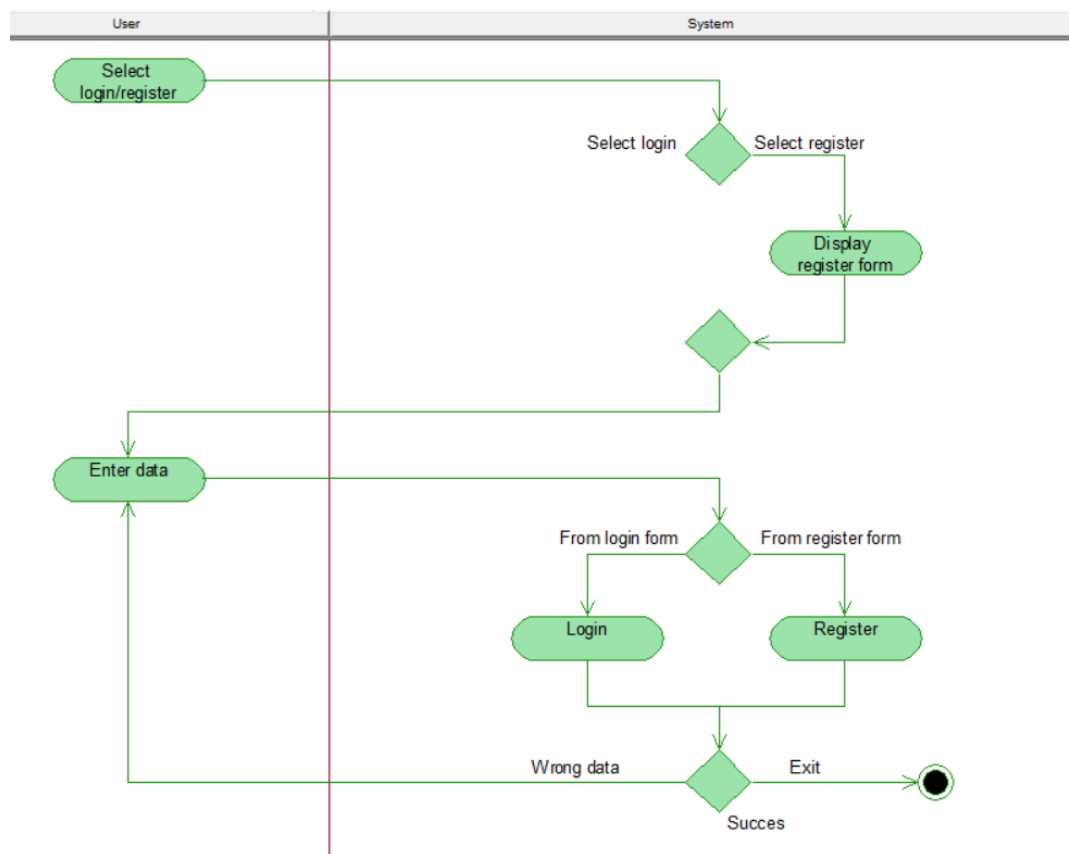


Рисунок 2.11 – Діаграма діяльності «Register/Login»

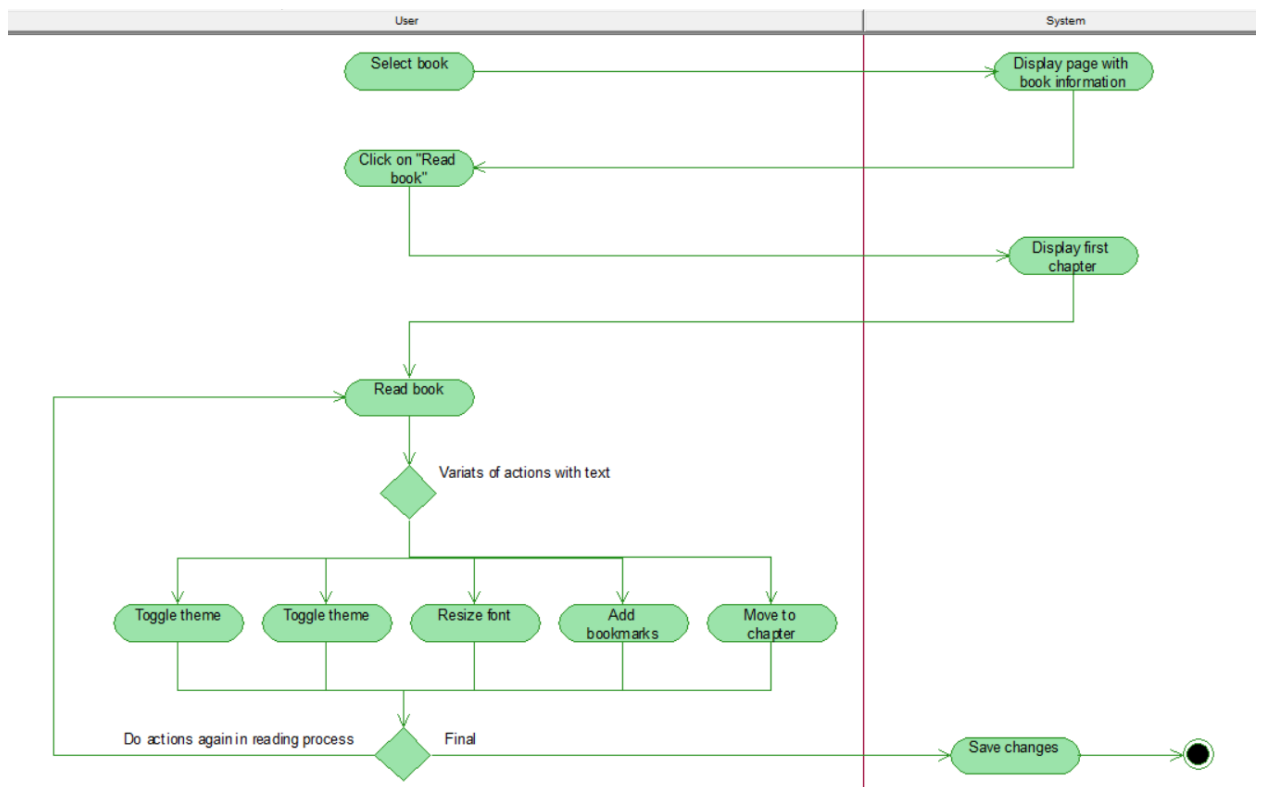


Рисунок 2.12 – Діаграма діяльності «Open book for reading»

2.6 Діаграма компонентів

Діаграма компонентів загалом описує фізичну архітектуру системи. Вона показує, як система розбита на структурні блоки та як вони взаємодіють між собою (див. рис. 2.13). Клієнтська та серверна частини представлені окремими модулями, компонентами яких відповідно слугують сторінки та узагальнені компоненти логічної частини застосунку. Слід зазначити, що у якості інтерфейсів у більшості випадків виступають конкретні дані, об'єкти в якості контрактів взаємодії. Також модуль серверної частини взаємодіє з базою даних.

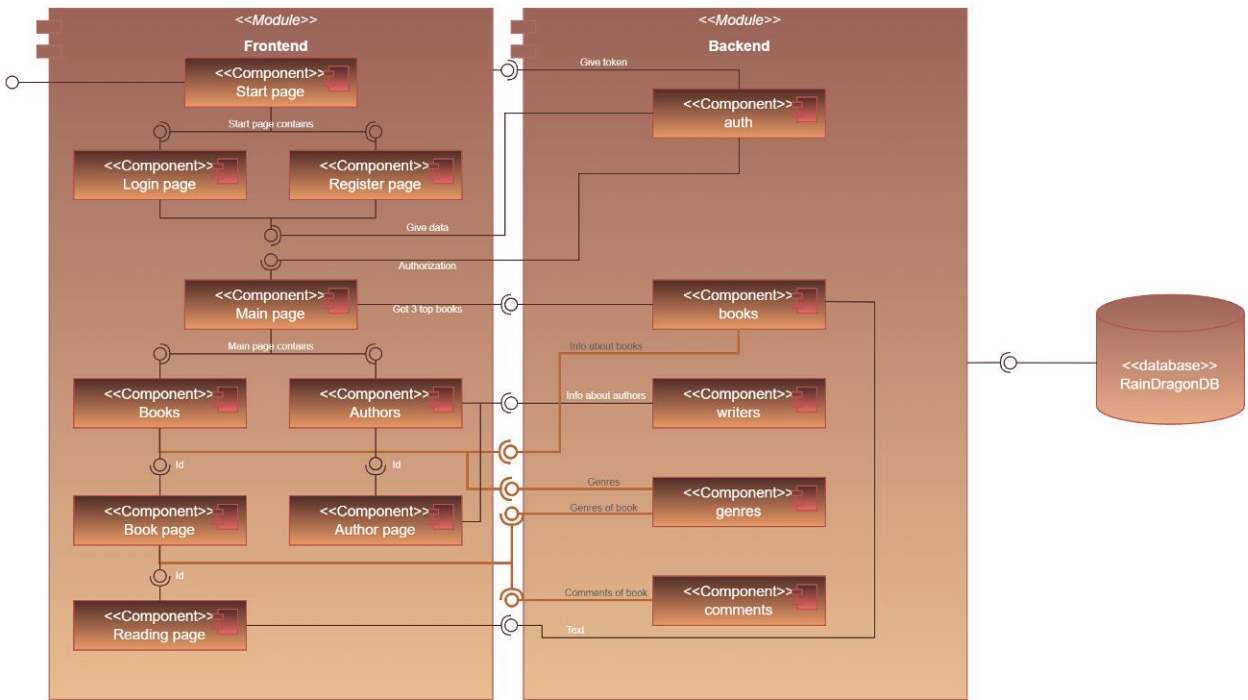


Рисунок 2.13 – Діаграма компонентів

2.7 Діаграма розгортання

Діаграма розгортання загалом завершує етап проектування та відображає фізичне розгортання системи (див. рис. 2.14). Вона представлена вузлами у якості клієнта (веббраузер), серверу (основа застосунку) та базою даних (PostgreSQL). Наявна також специфікація з приводу потрібного обладнання, операційної системи, тощо.

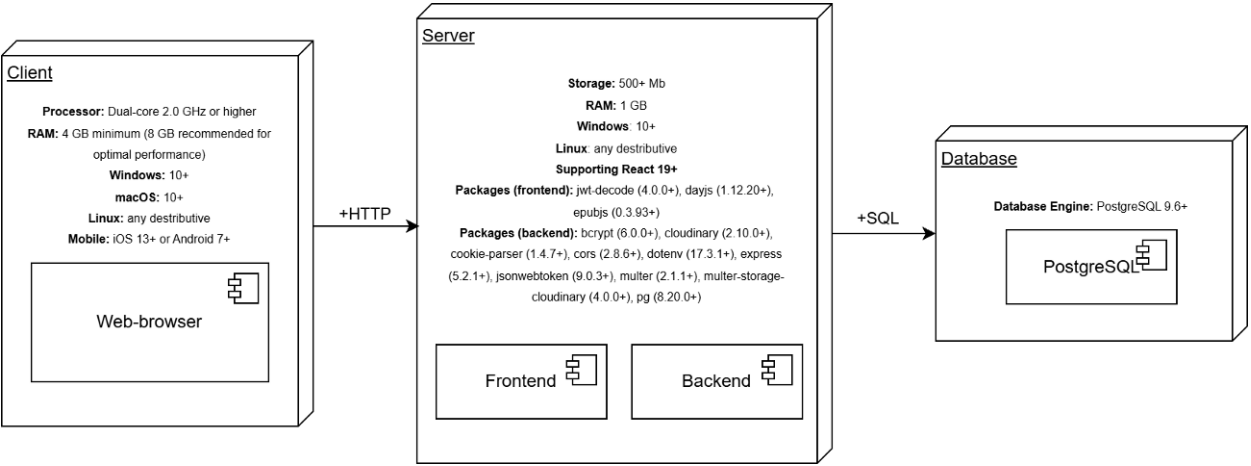


Рисунок 2.14 – Діаграма розгортання

3 РОЗРОБКА ТА ТЕСТУВАННЯ СИСТЕМИ

3.1 Загальний опис розробки інтерфейсу

Для оформлення зовнішнього вигляду сайту була обрана палітра спокійних та теплих тонів, які зазвичай асоціюються з комфортом та культурою Далекого сходу, традиційним живописом, письмом, тощо (див. рис. 3.1). Вони також не напружують зір, що дозволить зручно проводити багато часу на сайті.

```
:root {
  --text: #000000;
  --text-h: #08060d;
  --bg: #FAF2E7;
  --border: #A0735E;
  --code-bg: #D4C1B2;
  --accent: #432112;
  --error: #B00020;
  --accent-bg: rgba(118, 65, 42, 0.321);
  --accent-border: rgba(130, 58, 27, 0.5);
  --social-bg: rgba(63, 40, 13, 0.5);
  --shadow:
    rgba(0, 0, 0, 0.1) 0 10px 15px -3px,
    rgba(0, 0, 0, 0.05) 0 4px 6px -2px;
```

Рисунок 3.1 – Палітра

Для створення логотипу [7] та отримання стокових зображень були використані безкоштовні онлайн-ресурси.

Натхненням на створення логотипу (див. рис. 3.2) слугувало традиційне використання чорнил у живописі та письмі в культурі Азії. Назва «Rain Dragon», що означає «Дощовий Дракон», посилається на персонажа китайської міфології Ін Луна [9], який є наймогутнішим з драконів і є божеством дощу. В поєднанні це створює інтуїтивну асоціацію застосунку з тематикою Азії, що посилюють відповідні шрифти.



Рисунок 3.2 – Логотип

3.2 Нетривіальні рішення

Проблемою виявилася **автентифікація** користувача у системі.

Логічним рішенням було використання токенів [6]. Система при кожному запиті має перевіряти наявність Access Token. Якщо токен прострочено — система має створити новий за допомогою Refresh Token, який довго зберігається. Всі запити мають відбуватися миттєво.

Функція *requestWithRefresh* замінює стандартний *fetch*, викликаючись замість нього (див. рис. 3.3). Всередині неї знаходиться звичайний *fetch*, що дозволяє зробити звичайний запит з токеном, за його наявності. Якщо подальша перевірка показує, що токен не було оновлено, то перший запит, який пройде цю перевірку, викличе зміну статусу оновлення, щоб інші запити відправилися до відповідної черги *refreshSubscribers*. Коли токен буде оновлено — запити черги зможуть отримати новий токен.

```

26 const requestWithRefresh = async (url, options = {}) => {
27   // Функція для виконання запиту з автоматичним оновленням токена при 401 помилці
28   const makeRequest = (token) => {
29     fetch(url, {
30       ...options,
31       headers: {
32         ...(options.headers || {}),
33         Authorization: `Bearer ${token || localStorage.getItem("token")}`,
34       },
35     });
36   };
37   // Виконуємо запит вперше з поточним токеном
38   let res = await makeRequest();
39
40   if (res.status === 401) {
41     if (!isRefreshing) {
42       isRefreshing = true;
43       try {
44         const refreshRes = await fetch("http://localhost:5000/api/refresh", {
45           method: "POST",
46           credentials: "include",
47         });
48
49         if (refreshRes.ok) {
50           const data = await refreshRes.json();
51           localStorage.setItem("token", data.accessToken);
52           isRefreshing = false;
53           // У кожний елемент масиву прокидаємо новий токен, щоб вони могли повторити свої запити з оновленим токеном по черзі
54           onRefreshed(data.accessToken);
55         } else {
56           throw new Error("Refresh failed");
57         }
58       } catch (err) {
59         isRefreshing = false;
60         localStorage.removeItem("token");
61         window.location.href = "/";
62         return res;
63       }
64     }
65
66     // Створимо проміс, який буде чекати, поки токен оновиться, і потім повторить оригінальний запит з новим токеном
67     const retryOriginalRequest = new Promise((resolve) => {
68       // resolve як кур'єр, що чекає. Залишаємо йому функцію subscribeTokenRefresh, яка буде викликана, коли токен оновиться. Вона отримає новий токен і виконає makeRequest
69       subscribeTokenRefresh((token) => {
70         resolve(makeRequest(token));
71       });
72     });
73     return retryOriginalRequest;
74   }

```

Рисунок 3.3 – Функція requestWithRefresh

На серверній частині кожний запит, що потребує авторизованого користувача, пропускається через *authMiddleware*, що перевіряє наявність Access Token та відповідає помилкою 401 у разі його відсутності (див. рис. 3.4).

```

3 export const authMiddleware = (req, res, next) => {
4   const authHeader = req.headers.authorization;
5   const token = authHeader?.split(" ")[1];
6
7   if (!token) return res.status(401).json({ message: "Токен відсутній" });
8
9   try {
10    const decoded = jwt.verify(token, process.env.JWT_SECRET);
11    req.user = decoded;
12    next();
13  } catch (err) {
14    // Якщо токен прострочений або невалідний – просто 401
15    // Клієнт отримає цю помилку і зрозуміє, що треба йти на /refresh
16    return res.status(401).json({ message: "Token expired or invalid" });
17  }
18 };

```

Рисунок 3.4 – authMiddleware

Функція *refresh* дозволяє взяти Refresh Token з cookie сайту (див. рис. 3.5). Завдяки декодуванню можна відновити інформацію про *userId* та *admin* статус та повернути новий Access Token.

```

46 export const refresh = async (req, res) => {
47   try {
48     const refreshToken = req.cookies?.refreshToken;
49     if (!refreshToken) return res.status(401).json({ message: "No refresh token" });
50
51     const decoded = jwt.verify(refreshToken, process.env.REFRESH_SECRET);
52
53     // Об'єднуємо userId та admin в один об'єкт payload
54     const newAccessToken = jwt.sign(
55       { userId: decoded.userId, admin: decoded.admin },
56       process.env.JWT_SECRET,
57       { expiresIn: "1h" }
58     );
59
60     res.json({ accessToken: newAccessToken });
61   } catch (err) {
62     res.status(403).json({ message: "Refresh token invalid" });
63   }
64 };

```

Рисунок 3.5 – Функція refresh

3.3 Демонстрація застосунку

Далі розглянуто інтерфейс застосунку. Спочатку розглянемо авторизацію:

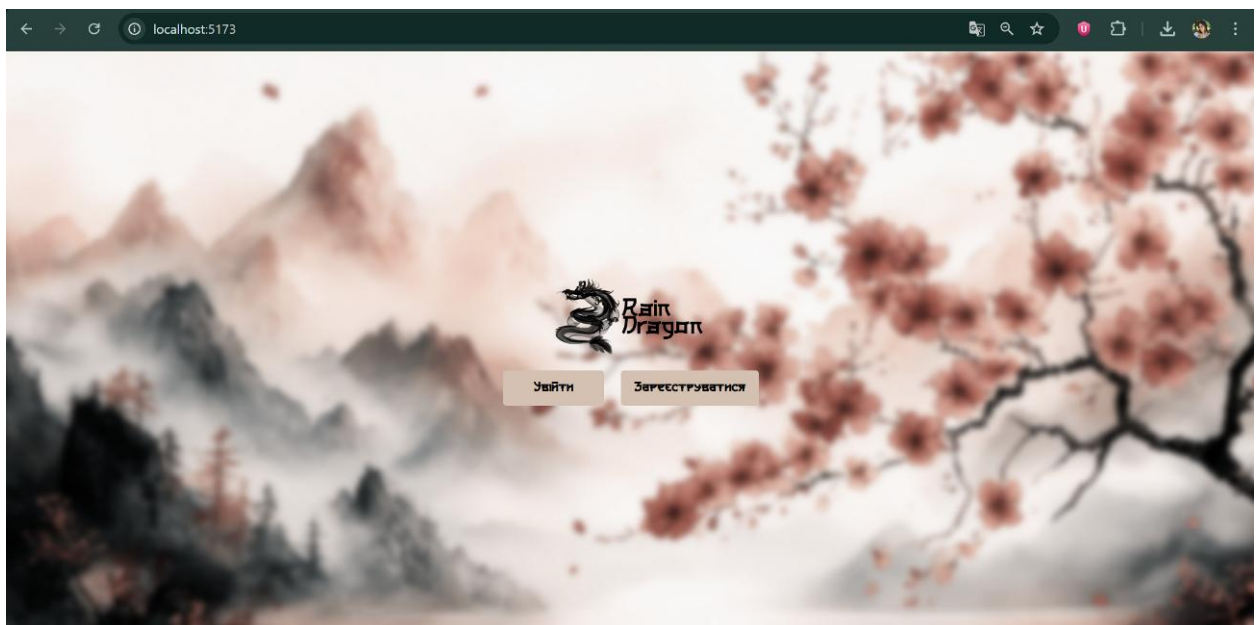


Рисунок 3.6 – Початкове вікно з пропозицією «Увійти» або «Зареєструватися»

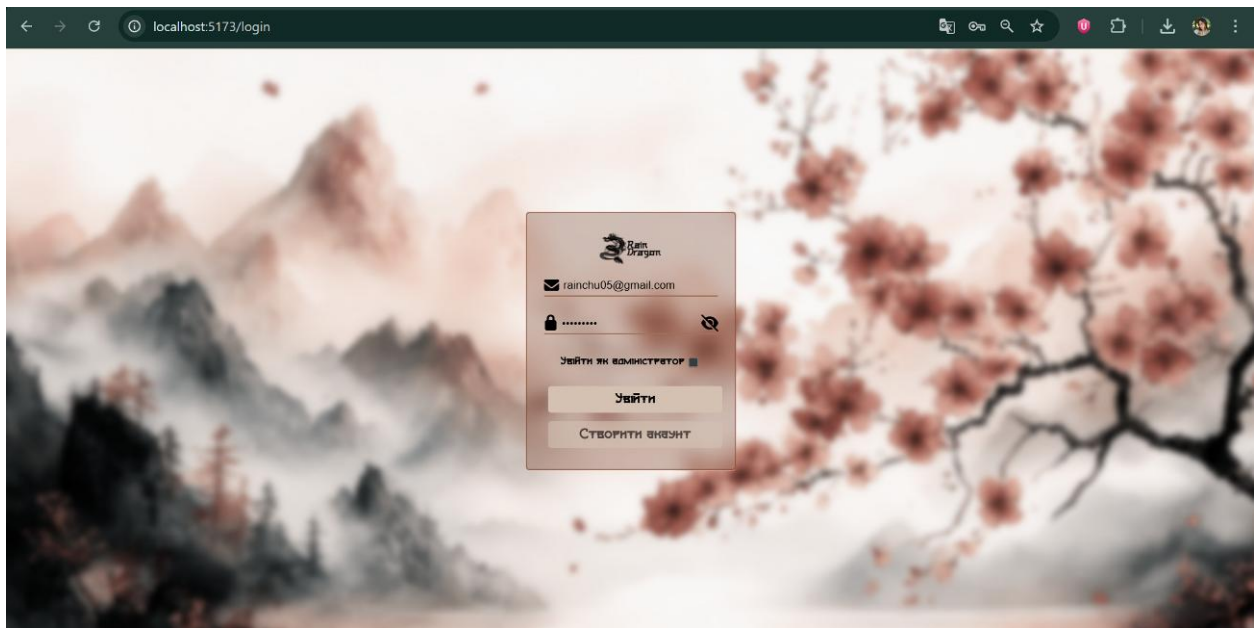


Рисунок 3.7 – Вікно «Увійти»

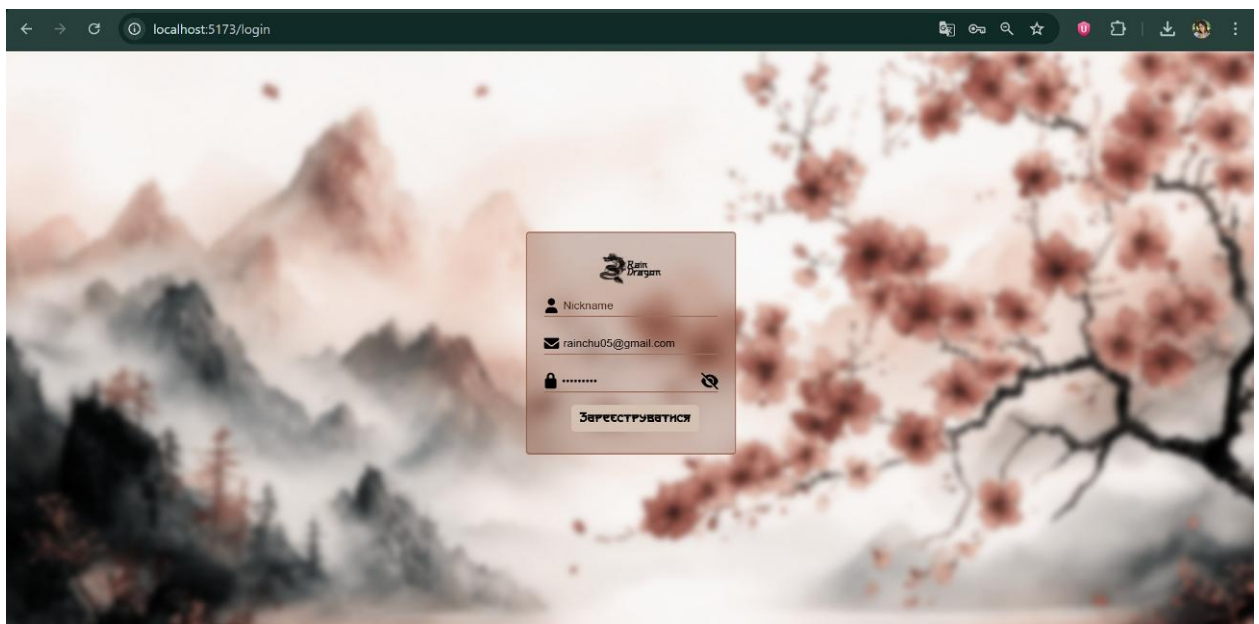


Рисунок 3.8 – Вікно «Зареєструватися»

Головна сторінка:

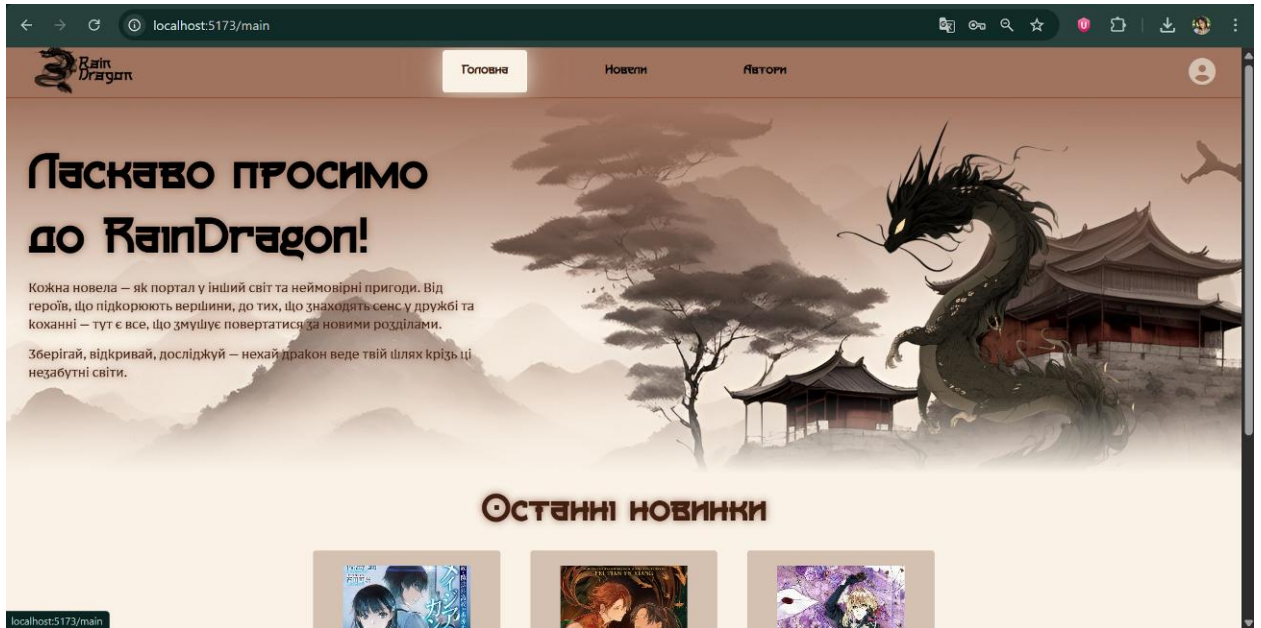


Рисунок 3.9 – Головна сторінка та банер

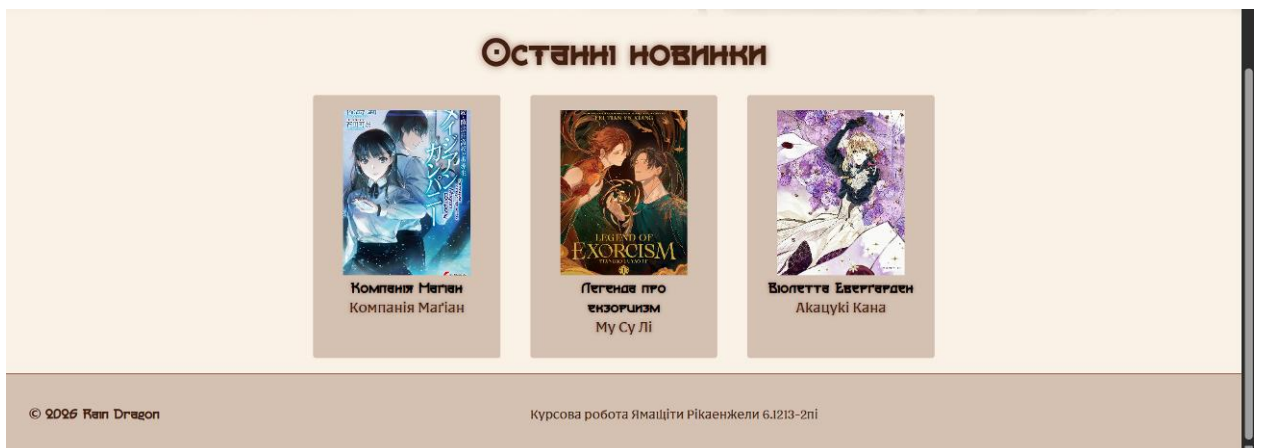


Рисунок 3.10 – Головна сторінка та останні додані книги

Переходимо розгляду загальної частини про книги:

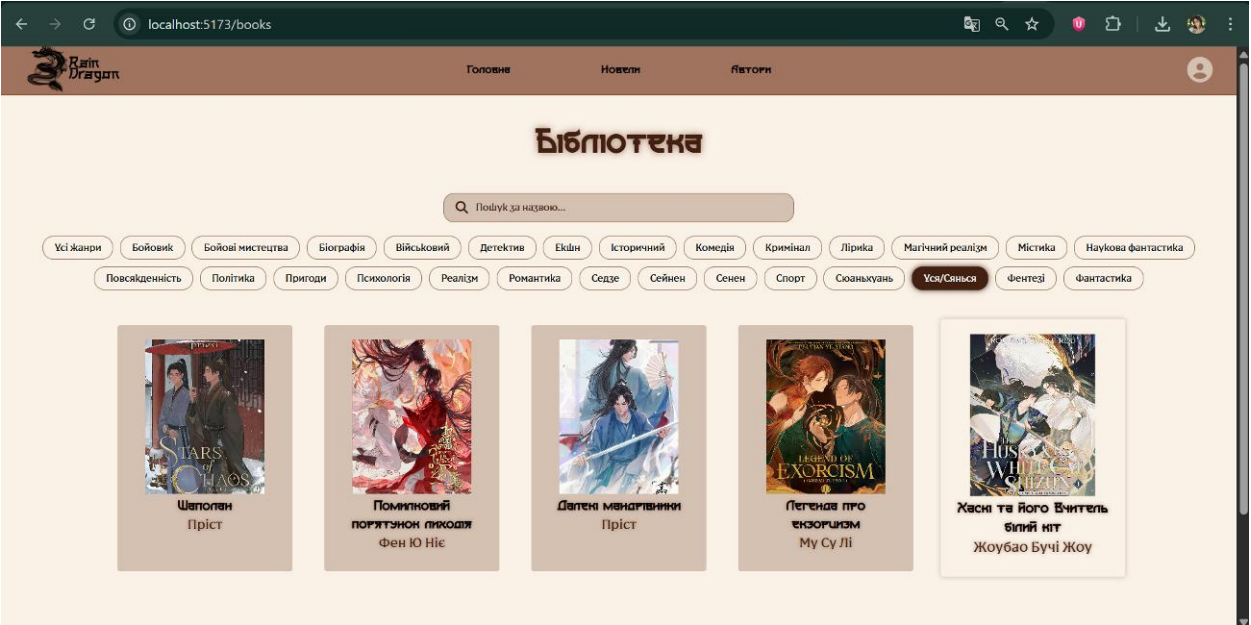


Рисунок 3.11 – Сторінка «Книги», де відбулася фільтрація за жанром «Уся/Сянься». Обрана книга підсвічується при наведенні

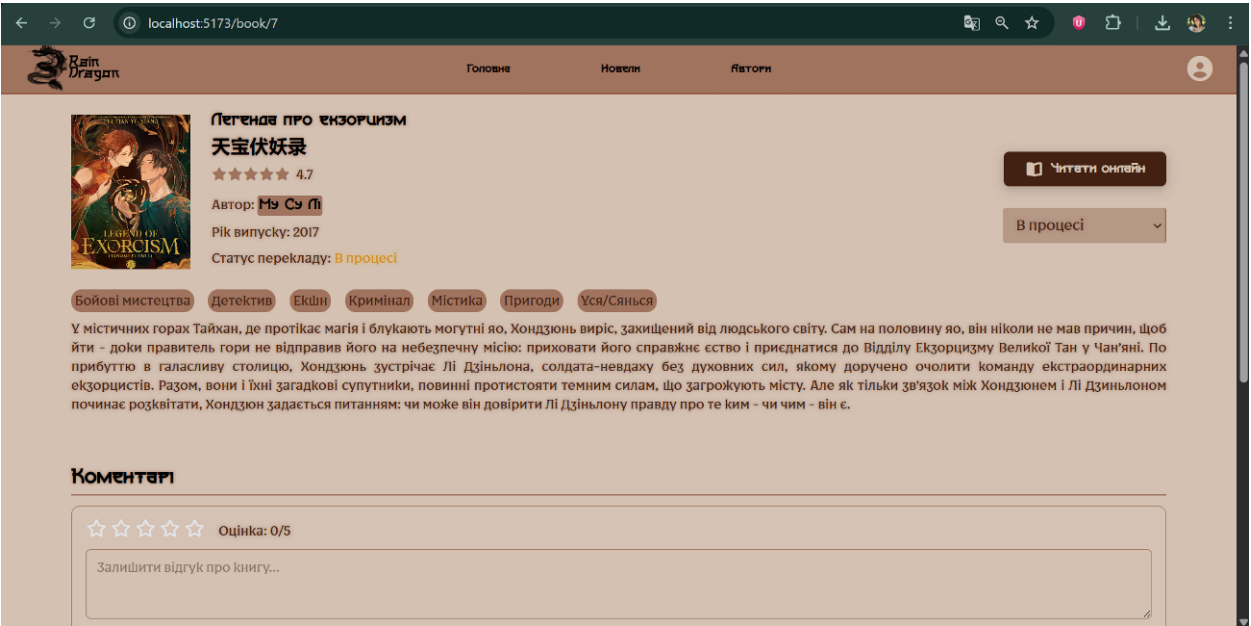


Рисунок 3.12 – Сторінка книги з інформацією

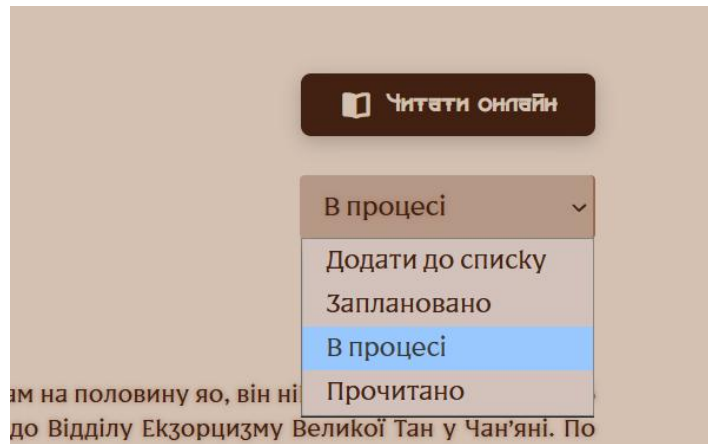


Рисунок 3.13 – Можливість змінити статус



Рисунок 3.14 – Коментарі



Рисунок 3.15 – Відповідь

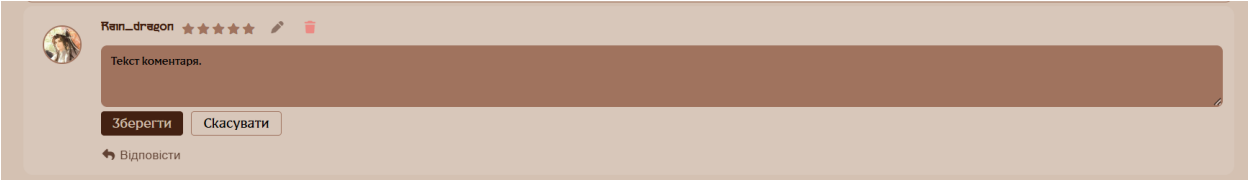


Рисунок 3.16 – Редагування свого створення

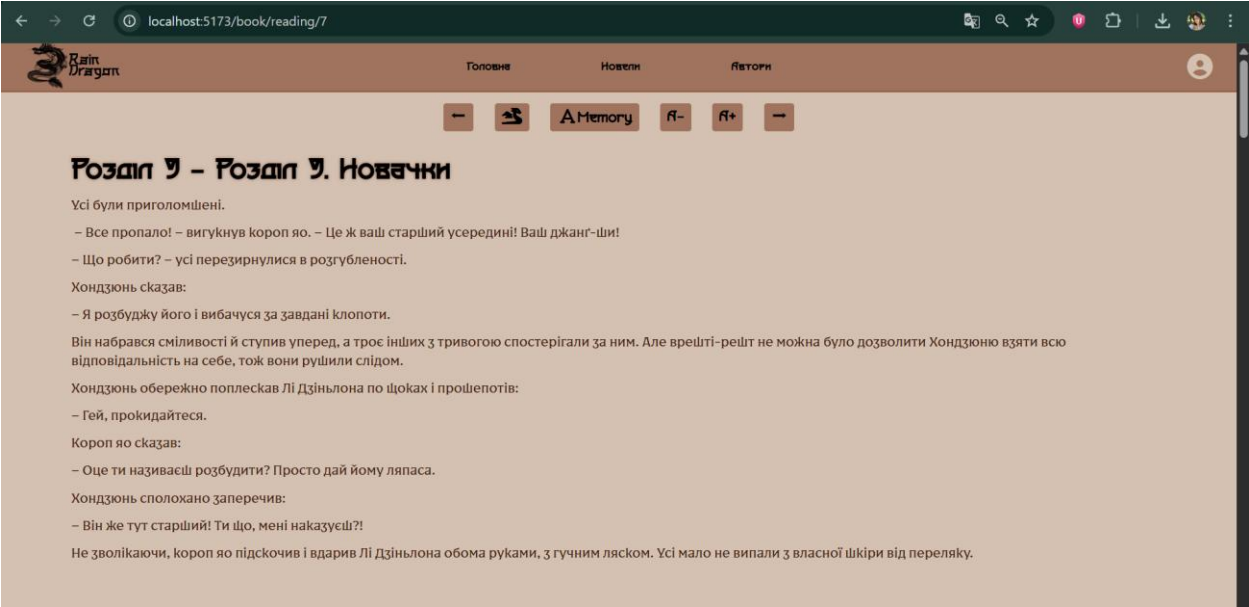


Рисунок 3.17 – Сторінка тексту книги

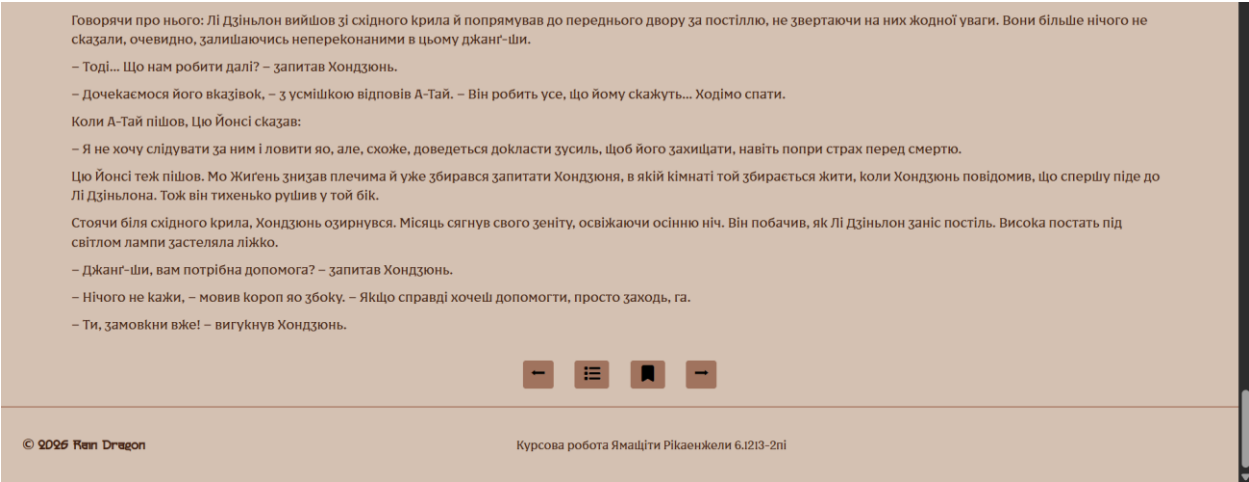


Рисунок 3.18 – Сторінка тексту книги

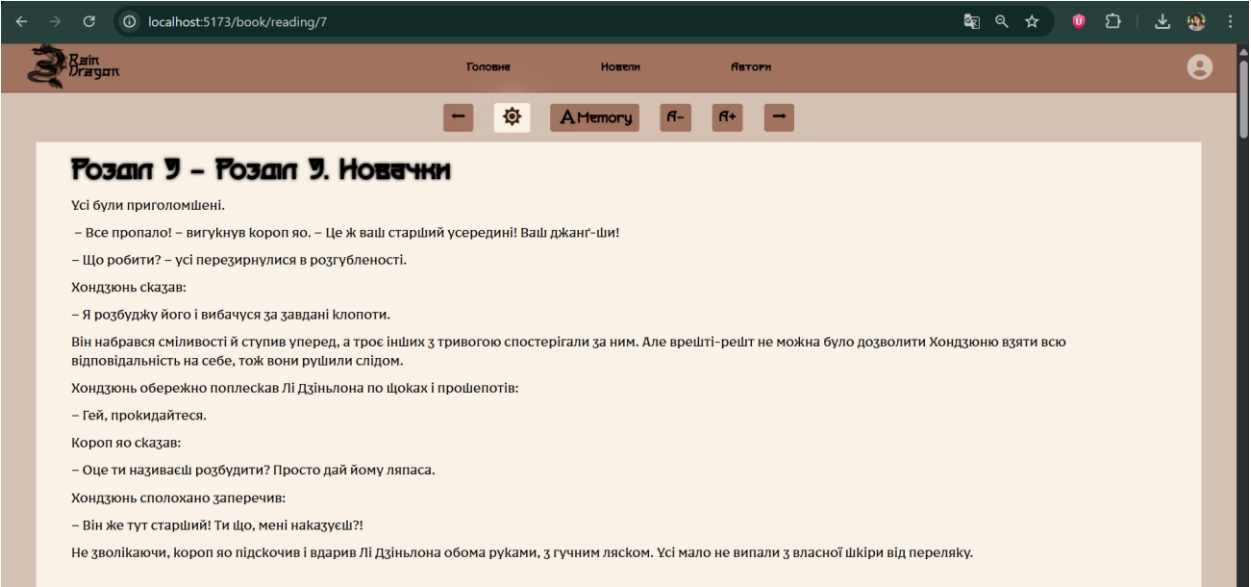


Рисунок 3.19 – Встановлення світлої теми

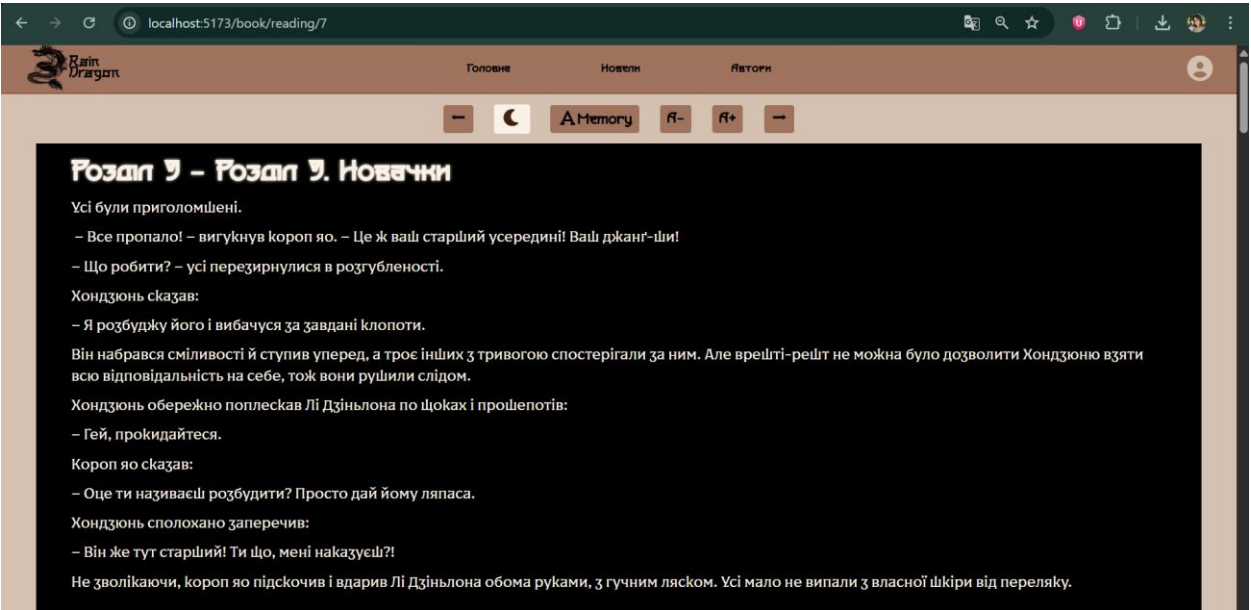


Рисунок 3.20 – Встановлення темної теми

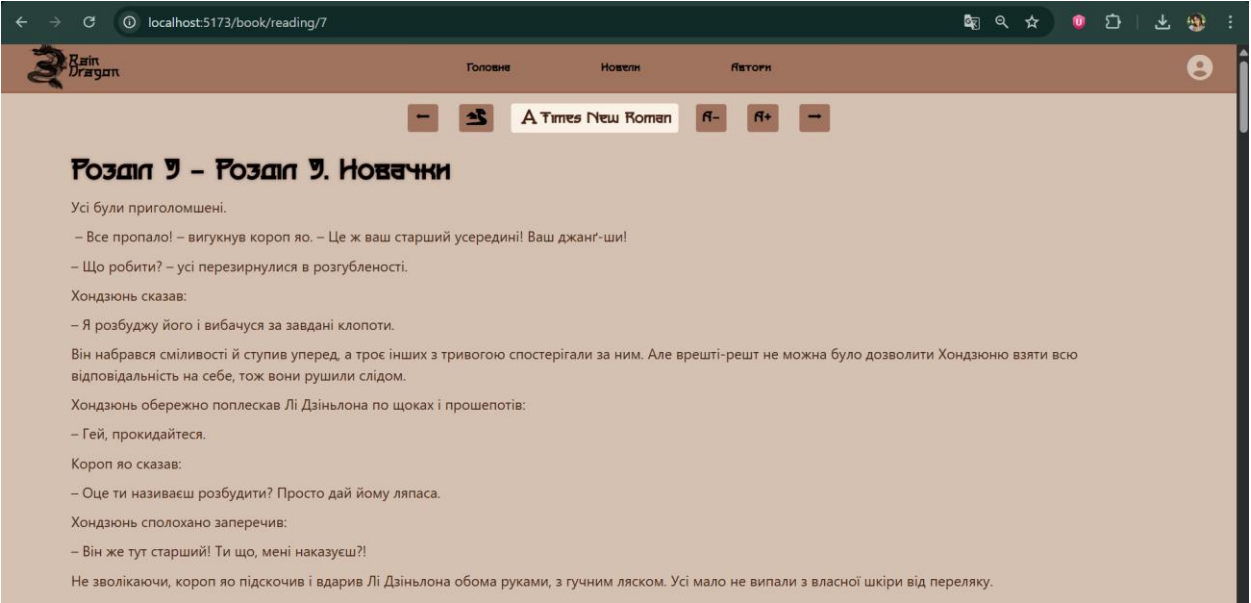


Рисунок 3.21 – Встановлення шрифту «Times New Roman»

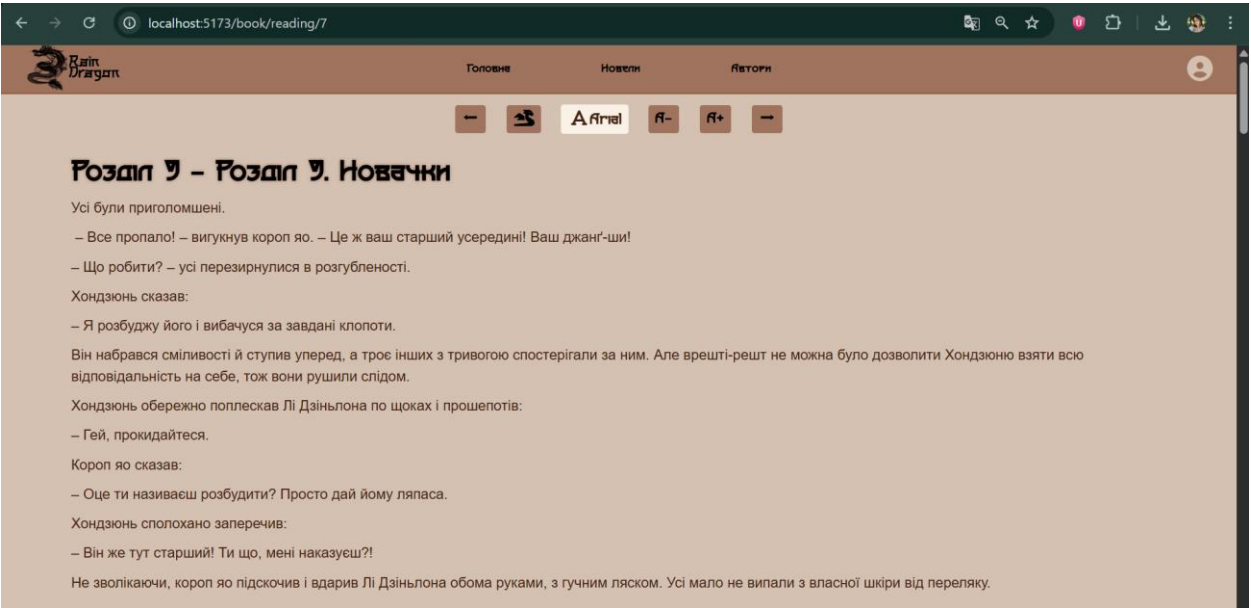


Рисунок 3.22 – Встановлення шрифту «Arial»

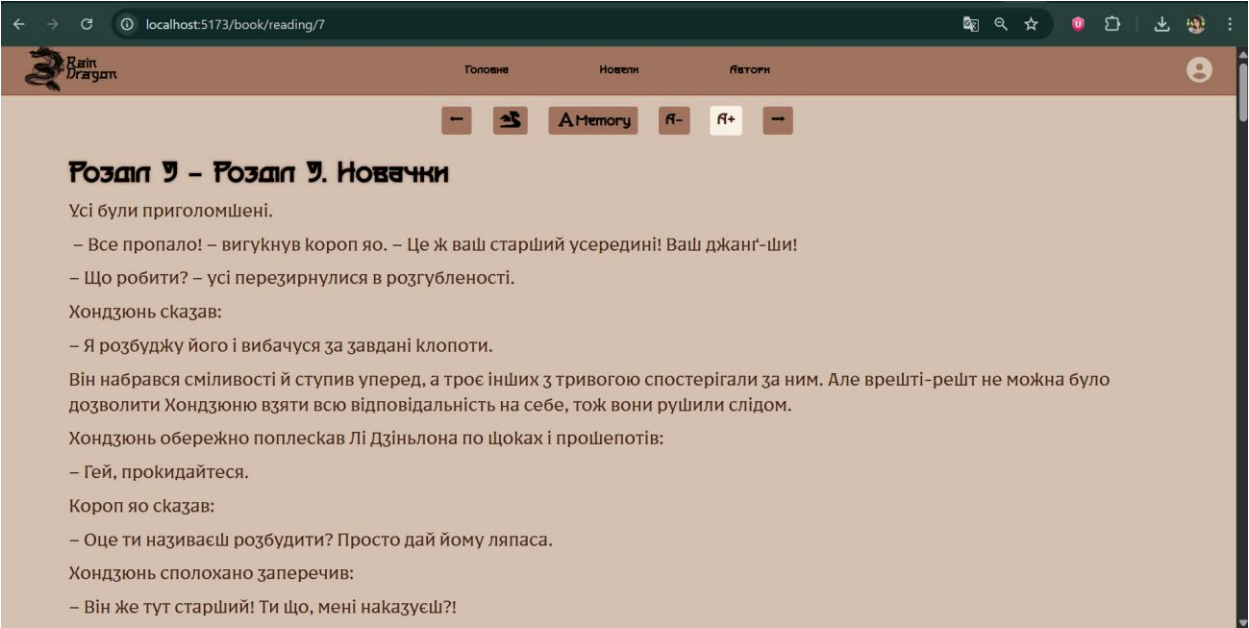


Рисунок 3.23 – Збільшення розміру шрифту

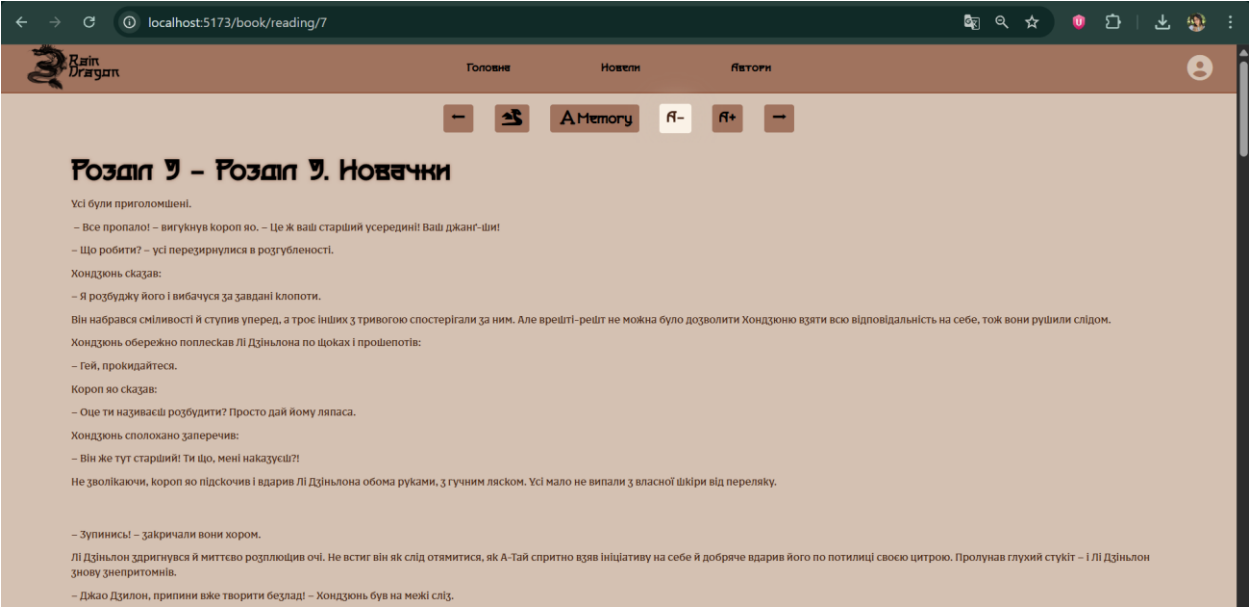


Рисунок 3.24 – Зменшення розміру шрифту

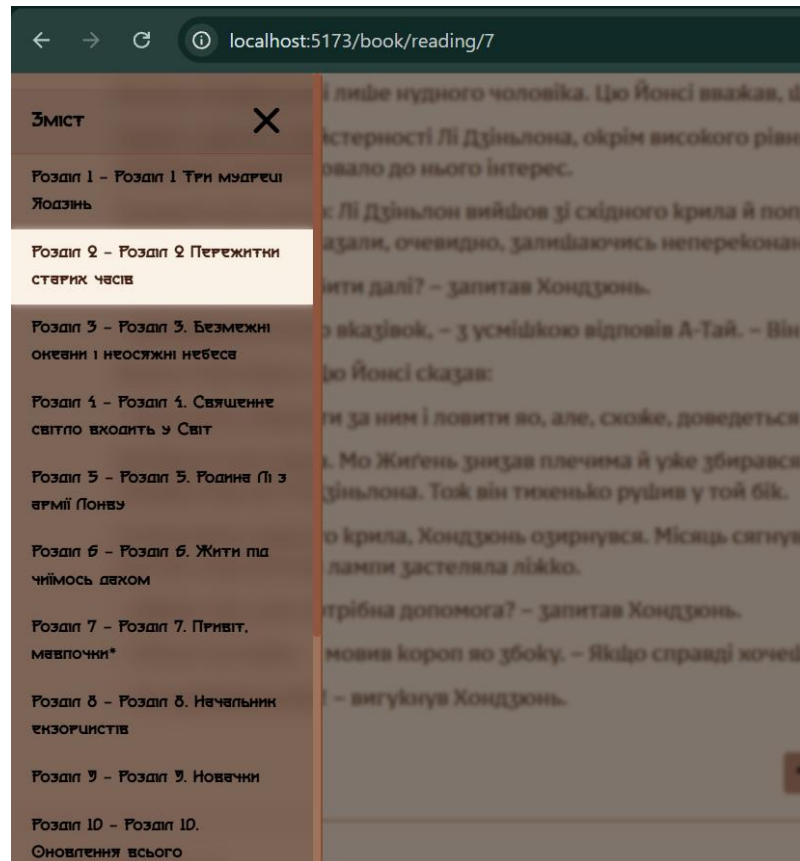


Рисунок 3.25 – Зміст книги. Обрана глава підсвічується

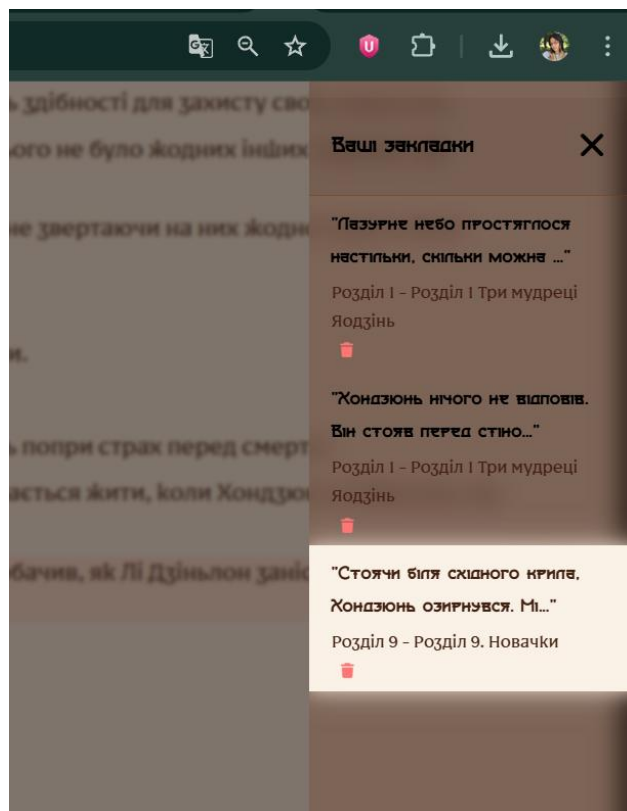


Рисунок 3.26 – Закладки. Обрана закладка підсвічується

Переходимо розгляду загальної частини про профіль користувача:

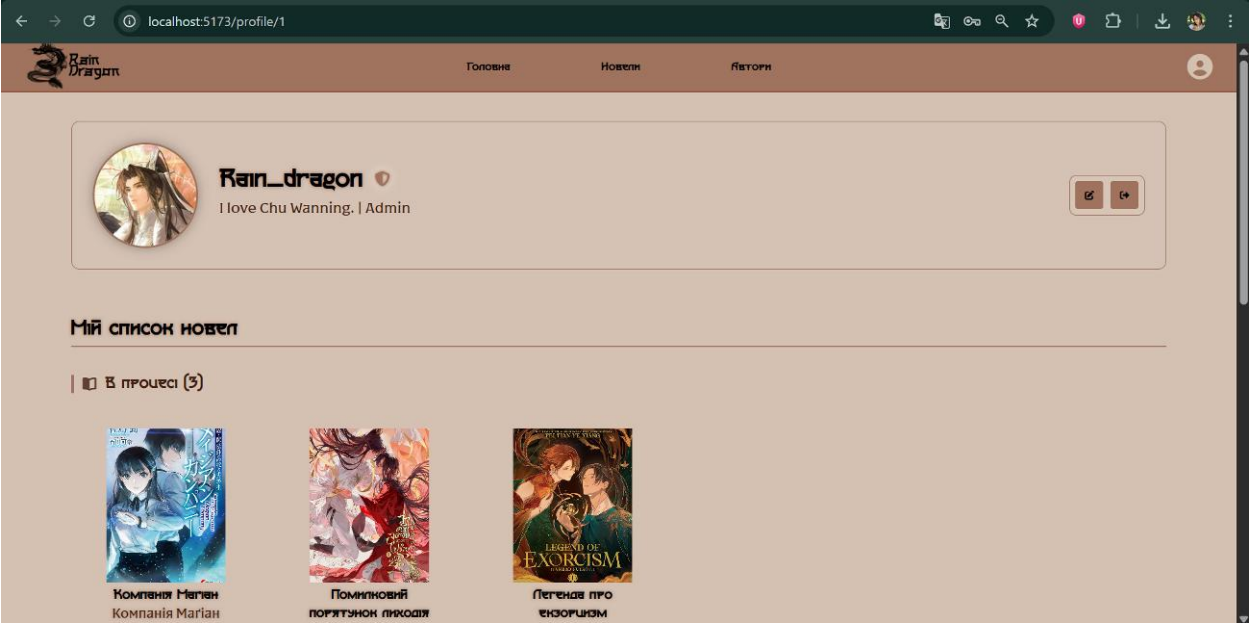


Рисунок 3.27 – Власний користувача

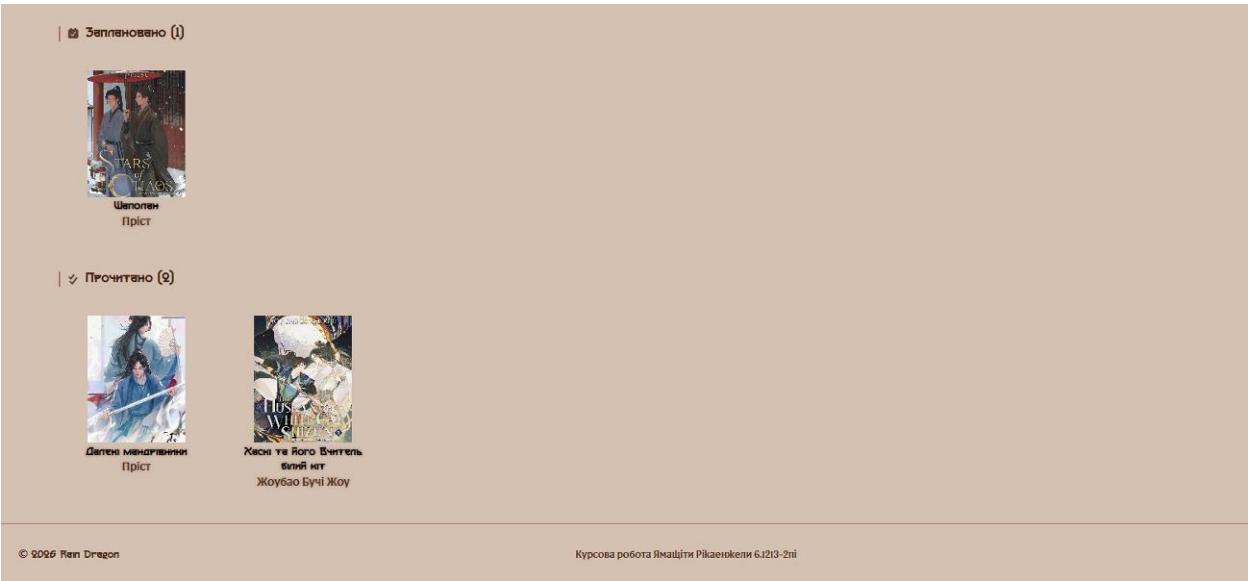


Рисунок 3.28 – Власний користувача

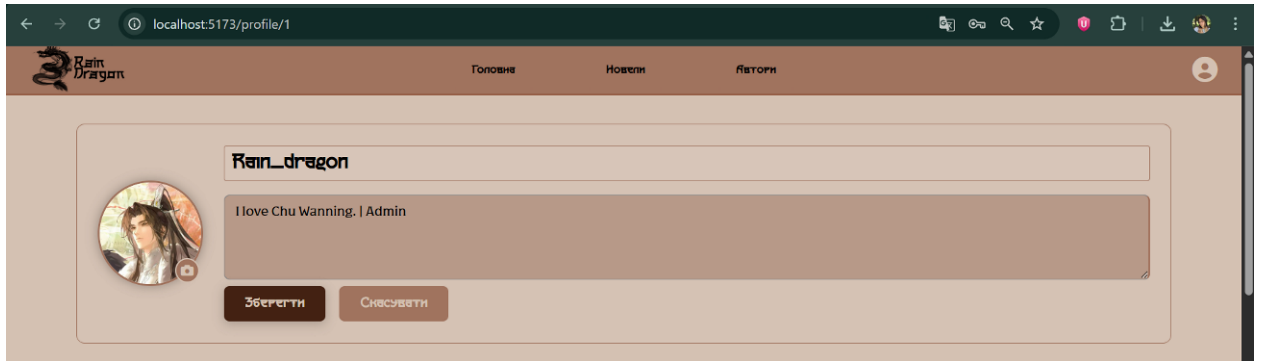


Рисунок 3.29 – Редагування профілю користувача

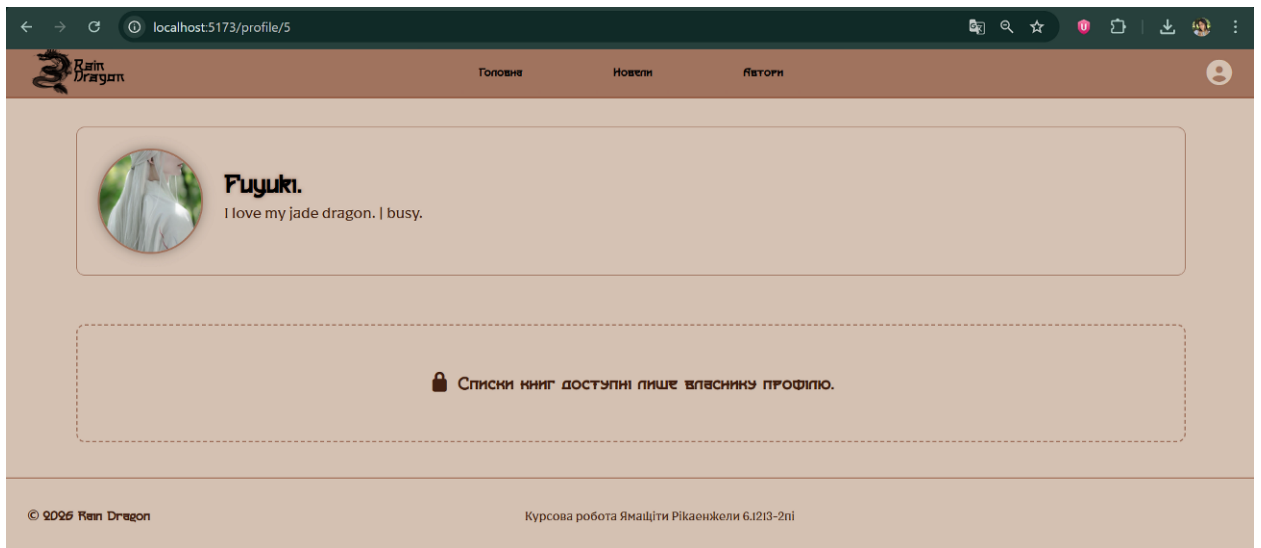


Рисунок 3.30 – Профіль стороннього користувача

Переходимо розгляду загальної частини про авторів:

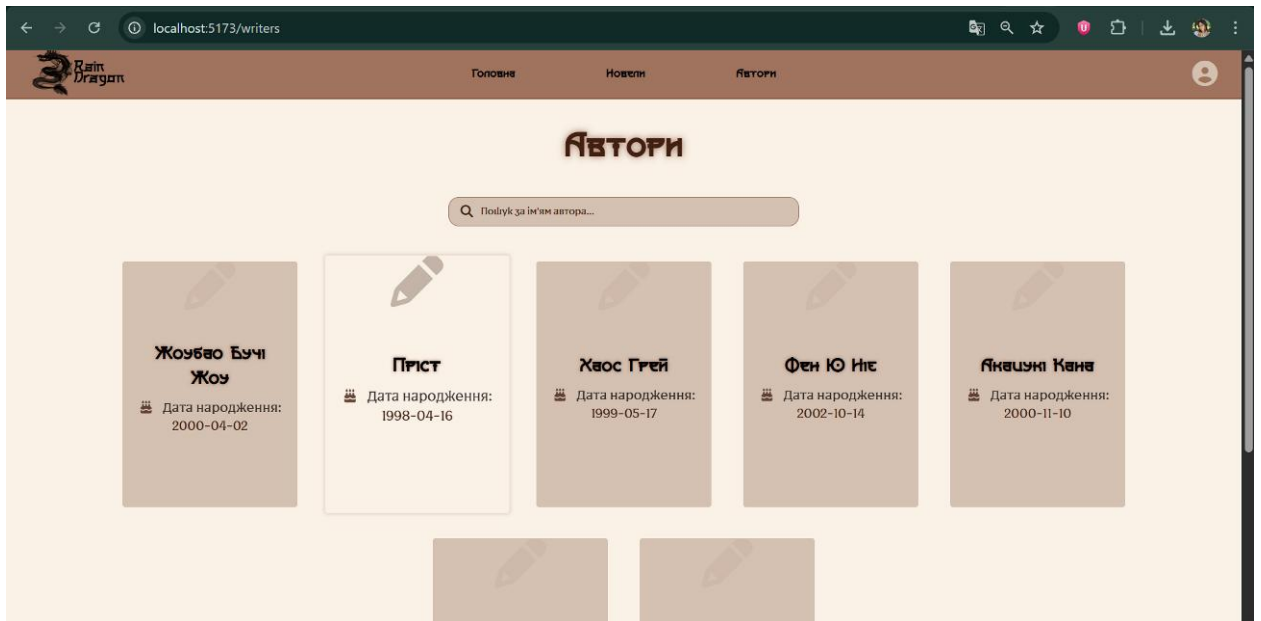


Рисунок 3.31 – Сторінка «Автори». Обраний автор підсвічується при наведенні

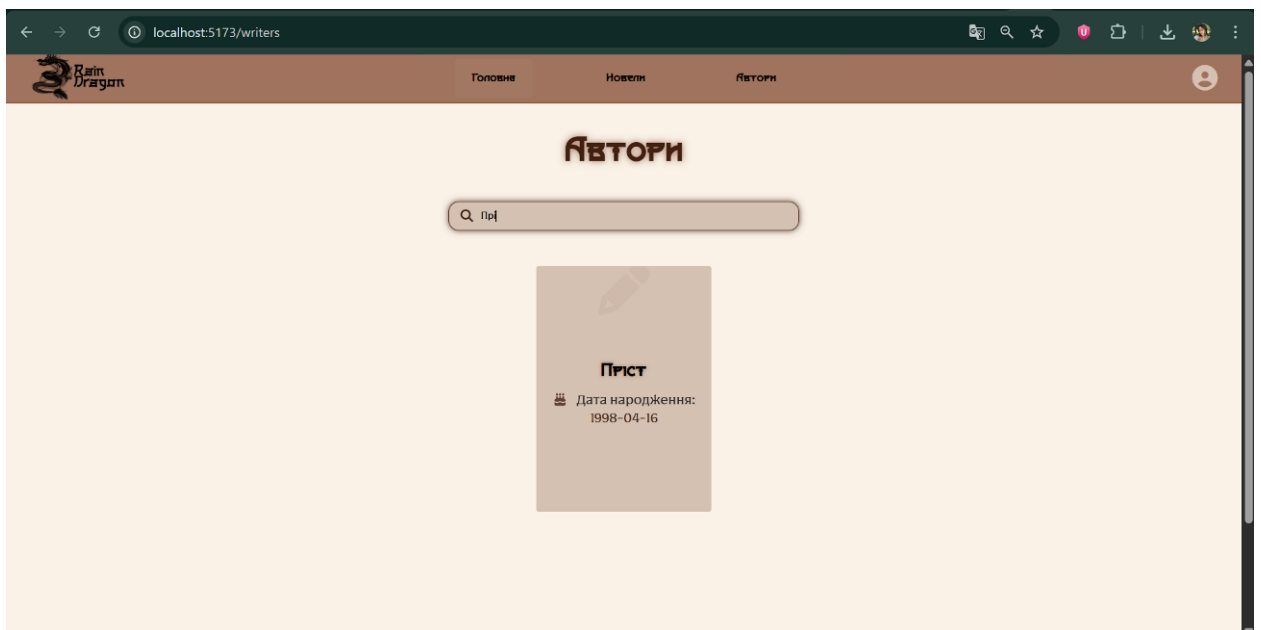


Рисунок 3.32 – Сторінка «Автори». Фільтрація за іменем

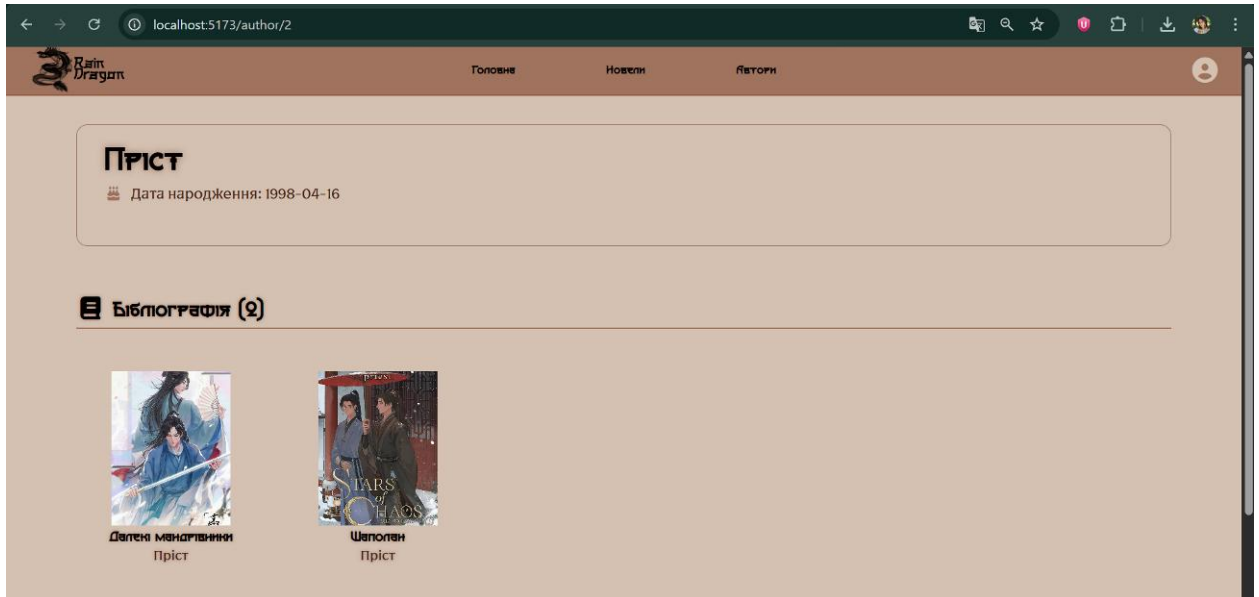


Рисунок 3.33 – Сторінка автору з інформацією та його книгами

Переходимо розгляду частини про додавання та редагування інформації:



Рисунок 3.34 – Форма для додавання книги

The screenshot shows a web browser window with the address bar displaying 'localhost:5173/upload/book/'. The form is titled 'Кількість томів:' and contains several input fields: 'Кількість томів:' with the value '1', 'Кількість глав:' with the value '0', 'Екстри:' with the value '0', and 'Опис книги:' which is a large text area. Below these are two more fields: 'Обкладинка (Посилання на Google Drive):' with the placeholder text 'Вставте посилання на зображення...' and 'Текстовий файл (Посилання на Google Drive):' with the placeholder text 'Вставте посилання на текстовий файл...'. At the bottom of the form is a large button labeled 'СТВОРИТИ КНИГУ'.

Рисунок 3.35 – Форма для додавання книги

The screenshot shows a web browser window with the address bar displaying 'localhost:5173/upload/writer/'. The page has a header with a logo 'Ваша Драга' and navigation links 'Головна', 'Новели', and 'Відео'. The main heading is 'Додати нового автора'. The form contains two input fields: 'Повне ім'я:' with the placeholder text 'Наприклад: Мосян Тонсю' and 'Дата народження:' with the placeholder text 'дд.мм.тггг'. At the bottom of the form is a large button labeled 'СТВОРИТИ'.

Рисунок 3.36 – Форма для додавання автора

The screenshot shows a web browser window with the address bar displaying 'localhost:5173/upload/book/1'. The page has a header with a logo and navigation links: 'Головна', 'Новели', and 'Історія'. The main heading is 'Редагувати книгу'. The form contains the following fields:

- Оригінальна назва книги:
- Українська назва книги:
- Рік видання:
- Статус перекладу:
- Автори:
- Жанри:

Below the form, there are tags for genres: 'Бойові мистецтва', 'Детектив', 'Пригоди', 'Романтика', 'Уся/Свесья'.

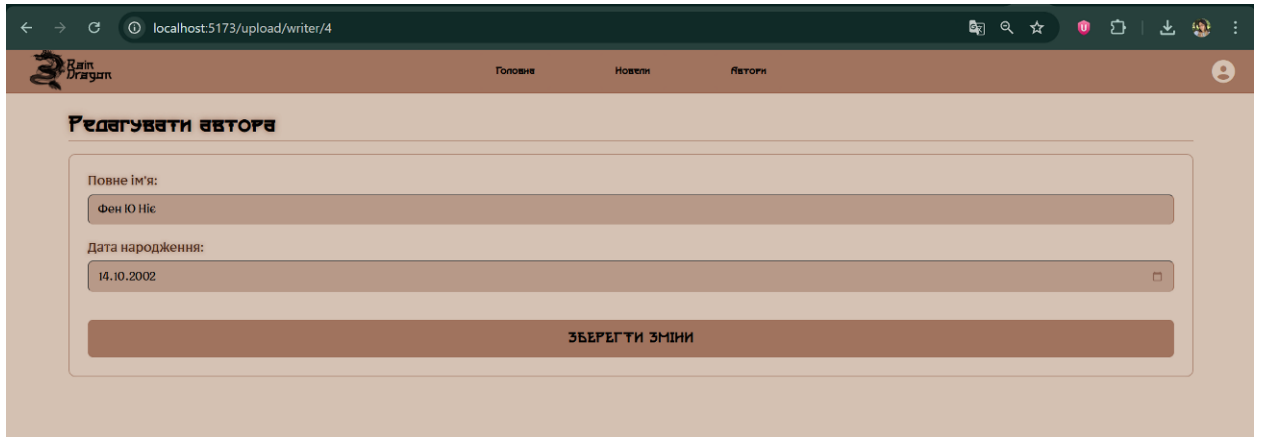
Рисунок 3.37 – Форма для редагування книги

The screenshot shows the same web browser window, but the form is now filled with data:

- Кількість томів:
- Кількість глав:
- Екстри:
- Опис книги:
- Обкладинка (Посилання на Google Drive):
- ✓ Посилання додано. [Переглянути за посиланням](#)
- Текстовий файл (Посилання на Google Drive):

At the bottom of the form, there is a button labeled 'ЗБЕРЕГТИ ЗМІНИ'.

Рисунок 3.38 – Форма для редагування книги



localhost:5173/upload/writer/4

Головна Новели Істори

Редагувати автора

Повне ім'я:
Фен Ю Ніє

Дата народження:
14.10.2002

ЗБЕРЕГТИ ЗМІНИ

Рисунок 3.39 – Форма для редагування автора

Оскільки в нефункціональних вимогах було зазначено «інтерфейс має бути адаптивним під різні розміри екрану», то продемонструємо адаптивність:



Рисунок 3.40 – Адаптивність початкового вікна

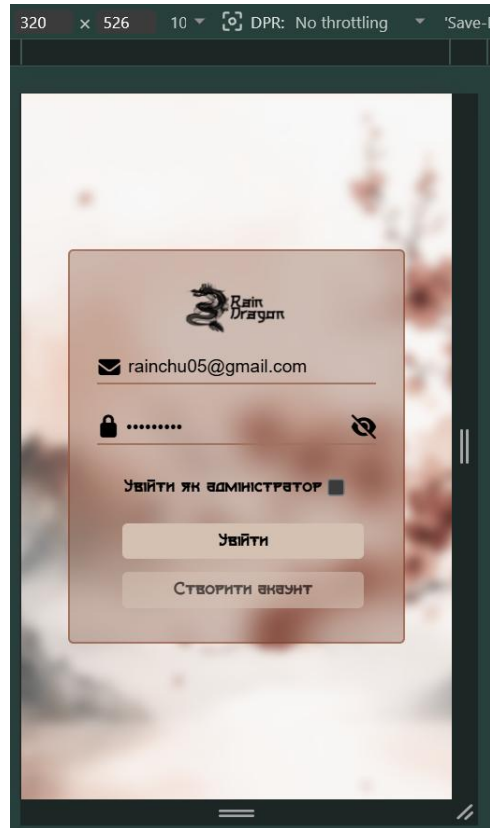


Рисунок 3.41 – Адаптивність вікна «Реєстрація» («Увійти») виглядає аналогічно

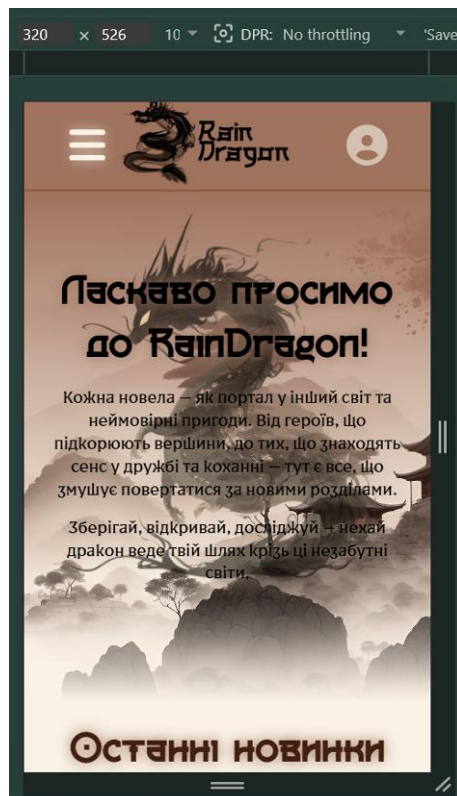


Рисунок 3.42 – Адаптивність головної сторінки

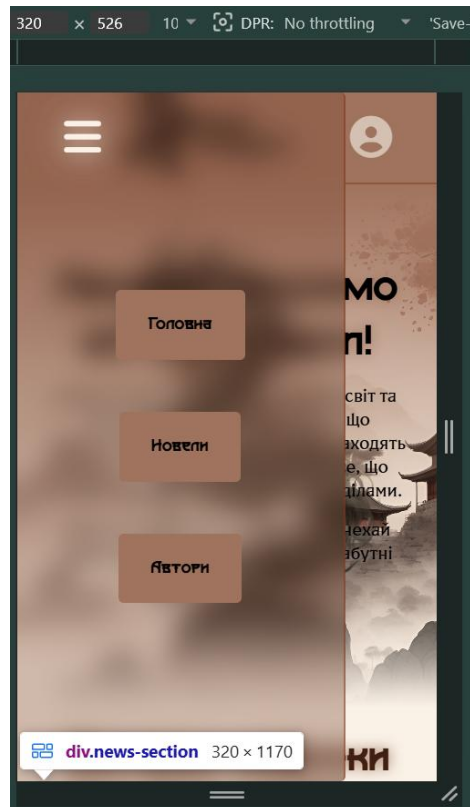


Рисунок 3.43 – Адаптивність навігації

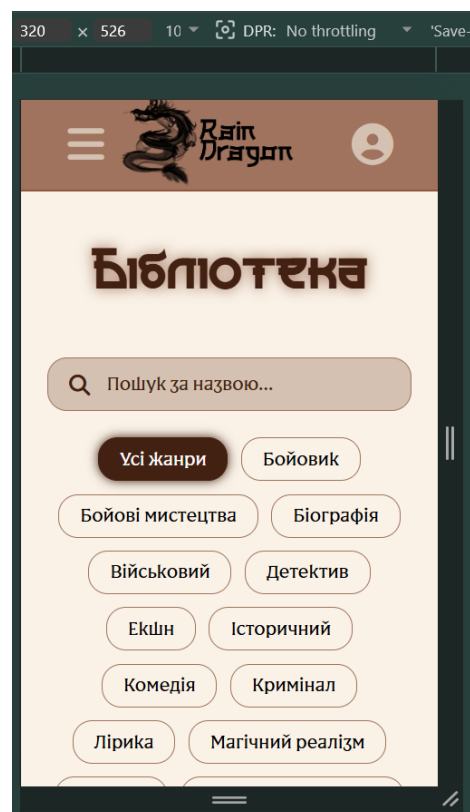


Рисунок 3.44 – Адаптивність сторінки «Книги»

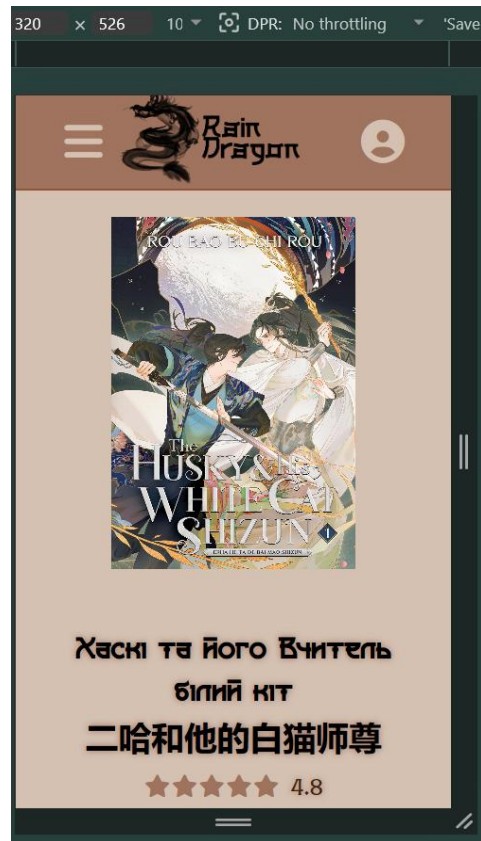


Рисунок 3.45 – Адаптивність сторінки книги з інформацією

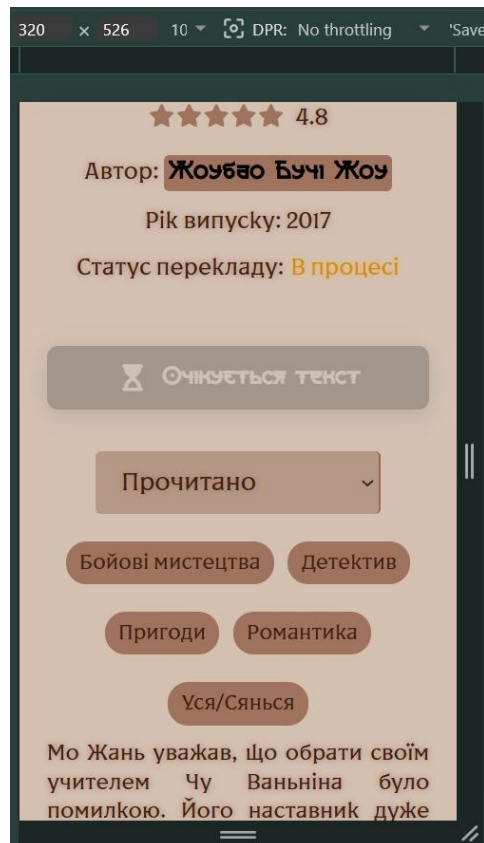


Рисунок 3.46 – Адаптивність сторінки книги з інформацією

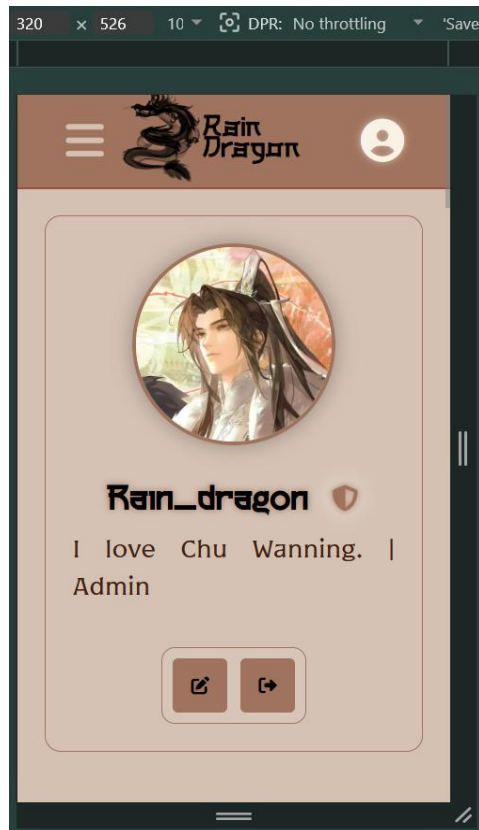


Рисунок 3.47 – Адаптивність профілю користувача

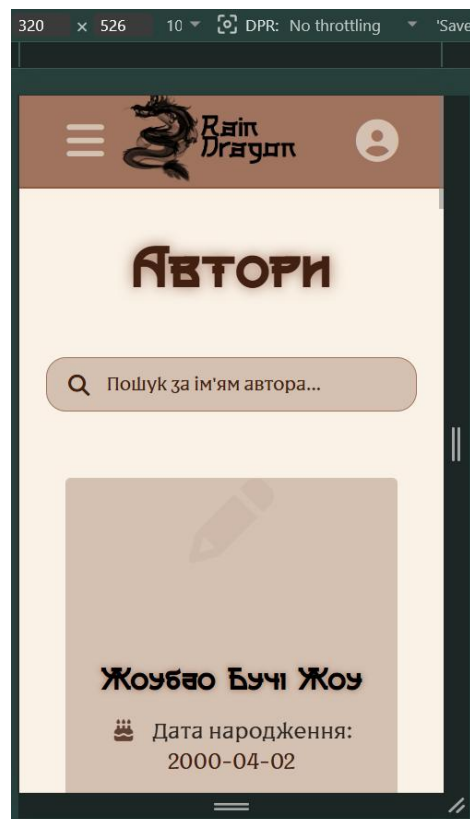


Рисунок 3.48 – Адаптивність сторінки «Автори»

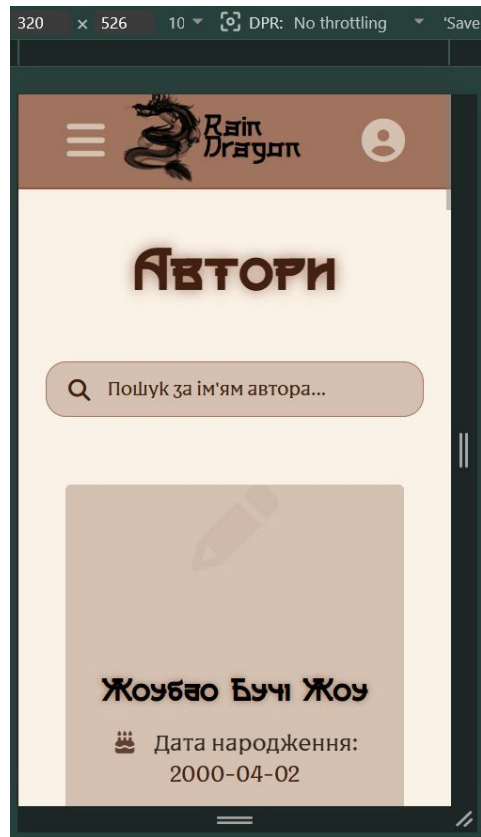


Рисунок 3.49 – Адаптивність сторінки «Автори»

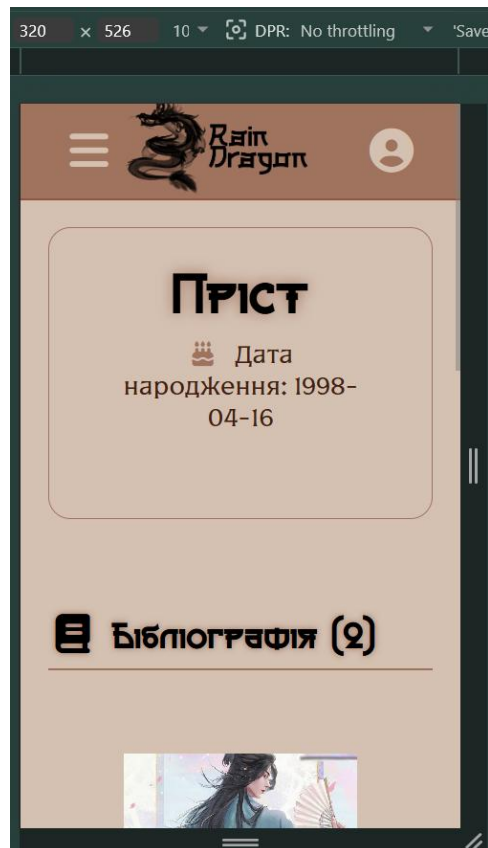


Рисунок 3.50 – Адаптивність сторінки автора з інформацією

3.4 Тестування застосунку

Було проведено юніт-тести, для яких знадобилися Vitest (двигун, який запускає тести [23]) та React Testing Library (інструмент, який допомагає «дивитися» на компоненти очима користувача [21]). Відповідні тест-кейси надані нижче:

Таблиця 3.1 – Тест-кейси

Назва тест-кейсу	Попередні умови	Кроки виконання	Очікуваний результат	Фактичний результат
Відображення вмісту книги	Наявність ID книги (123) та валідного ePub файлу.	1. Перейти за адресою /reader/123. 2. Дочекатися завантаження книги.	Відображено текст "Текст книги". Небажані теги (strong) видалено. Параграфам присвоєно ID та клас.	Отримано очікуваний результат
Перехід до деталей книги	Компонент BookCard завантажено з даними: ID 42, назва "Тестова Книга".	1. Натиснути на картку книги.	Викликано функцію навігації до сторінки /book/42.	Отримано очікуваний результат
Доступ адміна до створення книг	Користувач авторизований як адміністратор (admin: true).	1. Відкрити сторінку BooksPage.	На екрані присутня кнопка додавання книги з текстом "+".	Отримано очікуваний результат
Обмеження доступу для користувача	Користувач авторизований як звичайний юзер (admin: false).	1. Відкрити сторінку BooksPage.	Кнопка "+" відсутня на сторінці.	Отримано очікуваний результат
Успішна авторизація	Наявність вірних облікових даних (логін/пароль).	1. Ввести логін та пароль.	Токен збережено в localStorage. Відбувся перехід на	Отримано очікуваний результат

		2. Натиснути кнопку "Увійти".	сторінку /main.	
Авторизація з помилкою	Введено невірні облікові дані.	1. Ввести невірні дані. 2. Натиснути "Увійти".	З'явилося повідомлення про помилку: "Невірний логін або пароль".	Отримано очікуваний результат
Завантаження списку новинок	Сервер доступний та містить дані про топ книг.	1. Перейти на головну сторінку MainPage.	Відображено заголовок "Останні новинки" та список книг ("Шлях Дракона" тощо).	Отримано очікуваний результат
Обробка помилки сервера (500)	Сервер повертає помилку 500.	1. Відкрити головну сторінку.	У консоль виведено повідомлення про помилку сервера.	Отримано очікуваний результат

Всі тести пройшли успішно:

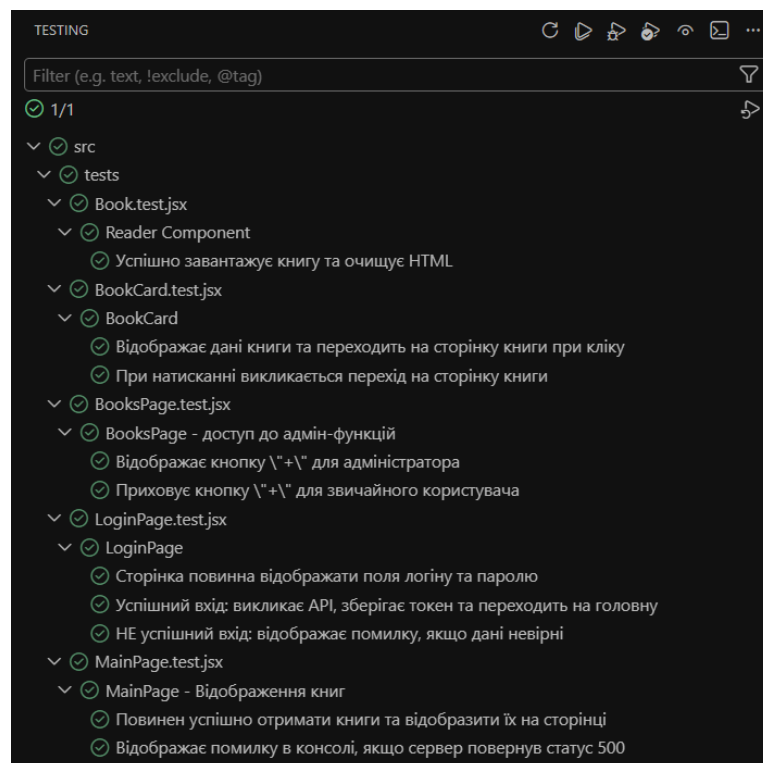


Рисунок 3.51 – Результати тестування

ВИСНОВКИ

У ході виконання курсової роботи було реалізовано вебзастосунок «RainDragon», призначений для зручного читання книг та збереження інформації про них та їх авторів. Додаток побудовано на основі сучасної архітектури з використанням СУБД PostgreSQL для збереження даних та вебтехнологій для клієнтської і серверної частин.

Було виконано повну роботу: визначення поставленої мети та аналізу предметної області, побудова ER-моделі, проектування за допомогою UML-діаграм, створення бази даних, заповнення таблиць, програмування запитів та застосунку.

В результаті, створена система може бути використана для вирішення проблеми зберігання інформації та аналізу існуючих перекладів азійських романів українсько мовою. Це дозволяє поглянути на проблему недостатньої кількості матеріалу серед україномовного суспільства, що може мотивувати удосконалювати вебзастосунок та розширювати його можливості.

ПЕРЕЛІК ПОСИЛАНЬ

1. Вебсайт з обраною предметною областю: <https://baka.in.ua/> (дата звернення: 14.03.2026)
2. Визначення CSS: <https://horoshop.ua/ua/glossary/css/> (дата звернення: 17.03.2026)
3. Визначення HTML: <https://engage.org.ua/shho-take-html-vycherpnyj-posibnyk-dlya-pochatkivcziv-i-profi/> (дата звернення: 17.03.2026)
4. Визначення нефункціональних вимог: <https://visuresolutions.com/uk/alm-guide/%D0%BD%D0%B5%D1%84%D1%83%D0%BD%D0%BA%D1%86%D1%96%D0%BE%D0%BD%D0%B0%D0%BB%D1%8C%D0%BD%D1%96-%D0%B2%D0%B8%D0%BC%D0%BE%D0%B3%D0%B8/> (дата звернення: 15.03.2026)
5. Визначення функціональних вимог: <https://studfile.net/preview/21632688/page:11/4> (дата звернення: 15.03.2026)
6. Використання токенів: <https://medium.com/@lakshyakumarsingh.2003.va/understanding-access-tokens-and-refresh-tokens-2ec4bc4f9a4f> (дата звернення: 28.03.2026)
7. Джерело зображень для створення логотипу: <https://pngtree.com/> (дата звернення: 25.03.2026)
8. Іконки для застосунку: <https://fontawesome.com/> (дата звернення: 26.03.2026)
9. Міф про Ін Луна: <https://www.ebsco.com/research-starters/history/yinglong-winged-dragon-chinese-myth> (дата звернення: 18.03.2026)
10. Поняття зручності у нефункціональних вимогах: <https://wezom.com.ua/ua/blog/funktsionalni-ta-nefunktsionalni-vimogi-do-programnogo-zabezpechennya> (дата звернення: 15.03.2026)

11. Порівняння популярності десктопних застосунків та вебзастосунків:
<https://wezom.com.ua/ua/blog/desktop-dodatok> (дата звернення: 17.03.2026)
12. Прийняті вимоги до інтерфейсу:
<https://studfile.net/preview/9049389/page:72/> (дата звернення: 15.03.2026)
13. Про Cloudinary: https://github.com/cloudinary/cloudinary_js (дата звернення: 19.03.2026)
14. Про CORS: <https://www.npmjs.com/package/cors> (дата звернення: 29.03.2026)
15. Про dotenv: <https://github.com/motdotla/dotenv> (дата звернення: 28.03.2026)
16. Про ePub.js: <https://www.npmjs.com/package/epubjs> (дата звернення: 17.03.2026)
17. Про Express.js: <https://expressjs.com/> (дата звернення: 26.03.2026)
18. Про JSON Web Token: <https://www.npmjs.com/package/jsonwebtoken> (дата звернення: 28.03.2026)
19. Про multer: <https://www.npmjs.com/package/multer> (дата звернення: 19.03.2026)
20. Про PostgreSQL Client: <https://node-postgres.com/> (дата звернення: 26.03.2026)
21. Про React Testing Library: <https://github.com/will-nemo/rtlcheatsheet> (дата звернення: 20.04.2026)
22. Про React: <https://dan-it.com.ua/uk/blog/chto-takoe-react-js-i-dlja-chego-on-nuzhen/> (дата звернення: 17.03.2026)
23. Про Vitest: <https://github.com/vitest-dev/vscode> (дата звернення: 20.04.2026)

ДОДАТОК А

Програмна реалізація

Клієнтська	частина	вебзастосунку:
https://github.com/YamashchitaEla/RainDragon_frontend.git		
Серверна	частина	вебзастосунку:
https://github.com/YamashchitaEla/RainDragon_frontend.git		